作品编号：CACR2026T4PMR7

作品类别：☑软件设计　☐硬件制作　☐工程实践

☐密码应用技术　☐其它

2026 年第十一届全国密码技术竞赛作品设计报告

作品题目：基于多类格困难假设的抗量子身份基匹配加密通用设计与优化实现

2026 年 10 月 21 日

中国密码学会

基本信息表

编号：CACR2026T4PMR7

作品题目：基于多类格困难假设的抗量子身份基匹配加密通用设计与优化实现

作品类别：☑ 软件设计 ☐ 硬件制作 ☐ 工程实践 ☐ 密码应用技术 ☐ 其它

作品内容摘要：

量子计算的发展对基于整数分解和离散对数问题的传统公钥密码提出了新的安全挑战，面向后量子环境的密码迁移已经成为网络与信息系统建设的重要问题。格密码具有多样的困难问题和丰富的结构形式，是抗量子公钥密码的重要技术路线。与此同时，证书体系下的加密与签名通常分别完成接收者访问控制和发送者身份认证；在机构定向投递、受控设备通信和来源受限的消息服务中，系统还需要在一次消息处理过程中同时核对通信双方的身份。身份基匹配加密正是面向这一需求：只有接收者身份与发送者指定的身份一致，并且发送者身份与接收者认可的身份一致时，消息才能被恢复。

本作品基于两层匿名分层身份基加密（HIBE）与身份基签名（IBS）的通用构造，设计并实现一套基于多类格困难假设的抗量子身份基匹配加密系统。系统将 HIBE 与 IBS 封装为彼此独立、可替换的密码后端，通过统一接口、能力声明和兼容性检查完成组合。HIBE 只封装 32 字节随机种子，应用消息由该种子派生的 AES-256-GCM 密钥保护，从而适配不同后端的明文容量；IBS 负责绑定发送身份和一次性验签公钥，外层一次性签名进一步认证完整密文字节。系统还实现了规范对象编码、固定身份发钥、状态持久化、异常拒绝和重启恢复。

作品完成 LATTE、Lai 两种 HIBE 与 BNN + ML-DSA、BNN + Falcon、Luo、SRGG、Sageloli 五种常规 IBS 的十种端到端组合，并将目标规模 Wang–PW IBS 接入 LATTE 组合。针对完整系统运行中暴露的计算与存储瓶颈，作品实现三项重点优化：利用多项式结构和逐频率低秩计算避免 Sageloli 采样中的稠密矩阵展开；按照字段实际取值范围对 SRGG 公钥、主状态、用户密钥和签名进行无损编码，降低对象保存与传输开销；为固定身份对建立 Lai 接收会话，将重复委派移出逐消息解密路径，降低连续通信中的逐消息成本。

十种常规组合在统一机器、输入与运行配置下全部完成，功能测试覆盖正确匹配、身份错配、密文篡改和新进程恢复。实验结果表明，不同 HIBE 主要改变接收状态、胶囊尺寸和解密成本，不同 IBS 主要改变发送者发钥、签名和发送密钥尺寸；三组代表方案由此分别面向低延迟交互、原生模格身份认证和固定身份对连续通信。软件提供命令行与浏览器两种操作入口，能够完成系统建立、双方发钥、消息发送、匹配解密、错误输入拒绝和状态恢复。该设计可用于机构定向消息、受控终端指令和跨部门协作等双方身份均受约束的通信场景，并为后量子身份密码的算法替换、部署选型与持续优化提供统一实现基础。

关键词（五个）：身份基匹配加密；格密码；分层身份基加密；身份基签名；格陷门采样

目录

第一章 作品概述 **1**
1.1 应用背景与研究意义 1
1.2 国内外研究现状 3
1.3 现有研究的工程实现问题 5
1.4 作品研究内容与技术路线 5
1.5 主要工作 6
1.6 报告结构 6

第二章 基础知识 **7**
2.1 数学符号与格基础 7
2.2 格上困难问题及结构变体 8
2.3 离散 Gaussian 与格陷门 9
2.4 身份基加密与分层身份基加密 10
2.5 身份基签名 10
2.6 身份基匹配加密与通用构造 11
2.7 本章小结 11

第三章 通用 IB-ME 系统设计 **12**
3.1 设计目标与总体架构 12
3.2 参与者、对象与身份顺序 12
3.3 原语接口与兼容性检查 13
3.4 IB-ME 五算法伪代码 14
3.5 基于短种子封装的混合加密 15
3.6 发送身份绑定与完整密文字节认证 16
3.7 规范编码与状态边界 17
3.8 正确性与异常处理 17
3.9 本章小结 17

第四章 算法实现与代表方案 **18**
4.1 分层身份基加密实现 18
4.2 身份基签名实现 19
4.3 困难假设、参数档与来源对应 21
4.4 已实现算法与组合矩阵 22
4.5 代表方案的性能指标与选取依据 22
4.6 三组代表的应用定位 24
4.7 本章小结 24

第五章 关键算法优化 **25**
5.1 Sageloli 结构化环采样优化 25
5.2 SRGG 对象的范围感知编码 27

5.3 Lai 接收会话优化 28
5.4 本章小结 29

第六章 软件系统与操作流程 30
6.1 分层软件架构 30
6.2 稳定的原语接口 31
6.3 组合配置与接入流程 31
6.4 固定发钥与持久状态 32
6.5 命令行与浏览器操作 33
6.6 运行优化与部署方式 34
6.7 本章小结 35

第七章 测试与结果分析 36
7.1 实验设置 36
7.2 十种常规组合的同机基本比较 37
7.3 代表方案的重复与专项分析 41
7.4 完整密文的通信开销 45
7.5 三项主要优化的实验结果 46
7.6 缓存与部署位置的完整流程对照 48
7.7 功能测试结果 49
7.8 本章小结 50

第八章 创新性与应用前景 51
8.1 作品创新性概述 51
8.2 应用需求与方案选择 52
8.3 机构定向消息与跨部门协作 52
8.4 受控终端与设备指令分发 53
8.5 固定身份对的连续通信 53
8.6 密码迁移与算法扩展 54
8.7 应用部署的实施条件 54
8.8 本章小结 54

第九章 总结与展望 55
9.1 工作总结 55
9.2 实验结论 55
9.3 后续工作 56

附录 A 实验与材料复现入口 57
A.1 数据、参数与源码位置 57
A.2 报告图表与 PDF 重建 58

参考文献 59

第一章　作品概述

1.1　应用背景与研究意义

1.1.1　机构定向通信的身份管理需求

由统一机构管理账号、岗位和设备身份的通信系统，一条消息往往同时受到接收权限和发送来源两类约束。以设备管理为例，运维中心向受控终端下发配置指令时，不仅要把内容限定给目标终端，还要让终端只接受其认可的运维身份。机构内部的定向文件投递、跨部门业务流转和来源受限的数据分发也有相同特点：通信双方的身份都参与消息能否被处理的判断，而不只是作为日志中的附加字段。

公钥加密与数字签名经过正确组合可以完成机密传输和来源认证，但两项身份关系通常由证书、账号目录、签名对象和加密对象分别表达。应用协议还要明确先签名还是先加密、签名覆盖哪些字段、证书怎样取得，以及身份不符时如何拒绝。同一业务规则因此分散在密码算法和应用逻辑的多个环节；只要身份表示或对象绑定不一致，就可能出现“密文发给了正确设备，但签名对应的并非接收端所认可角色”之类的语义偏差。

身份基密码为机构已经建立的身份管理体系提供了另一种表达方式。Shamir 提出以用户身份直接参与公钥密码运算的设想后，身份基加密、分层身份基加密和身份基签名分别发展出面向加密、分层发钥和来源认证的算法[2, 4, 5]。邮箱、机构账号或设备编号经规范编码后即可作为密码输入，可信发钥机构负责核验现实身份并签发私钥。该机制适合具有明确管理边界的组织，但也要求机构妥善保护主密钥并建立安全的私钥发放流程，并非对所有公钥基础设施的无条件替代。

1.1.2　双向身份匹配与通信关系保护

身份基匹配加密（Identity-Based Matchmaking Encryption，IB-ME）把上述双向身份条件纳入同一密码功能。发送者以自身身份 s 和目标接收身份 r 生成密文；接收者持有身份 r' 的接收密钥，并在解密时指定自己认可的发送身份 s'。只有 $r = r'$ 且 $s = s'$ 同时成立时，接收端才恢复消息。IB-ME 最初作为匹配加密在身份策略下的专门形式提出，用于在完成双向匹配的同时提供消息隐私、发送来源认证和通信身份保护[14]。

IB-ME 的意义不在于宣称普通加密与签名无法组合出相同业务功能，而在于把“谁可以接收”和“接收者认可谁发送”固定为一个密码语法。身份条件随密钥、密文和解密输入共同参与处理；当底层身份基加密具备匿名性时，公开信道上的观察者也不能直接从密文读出目标身份。图 1.1 以机构运维为例说明这种关系。可信发钥机构核验并发放身份密钥，发送端和接收端分别表达对方身份，公开网络只承载密文。

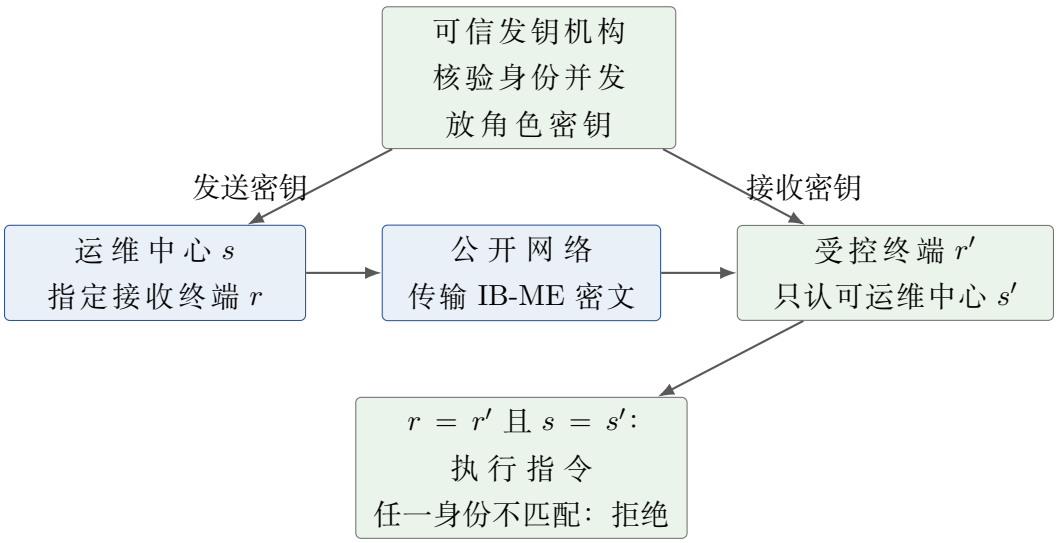


图 1.1 机构定向通信中的身份与密钥关系

这类能力能够对应到日常信息服务中的具体流程。例如，医院向患者发送检验报告或电子处方时，内容只应由指定患者读取，患者端同时需要确认消息来自指定医院或医生；学校向学生、家长发送定向通知时，接收者既要属于目标范围，也要确认发送方是相应学校或部门；智能设备接收配置或升级指令时，设备身份与控制中心身份同样需要双向核对。它还可用于跨部门受控文件流转，以及原始 IB-ME 研究讨论的隐私公告板等匿名而可认证的通信方式[14]。这些应用模式共享双向身份约束，但对在线延迟、密文长度、离线准备和连续通信成本的要求并不相同。因而，具体系统不仅需要一个可运行的 IB-ME 方案，还需要根据使用方式选择和替换底层原语。

1.1.3 后量子迁移下的格密码实现需求

公钥加密和数字签名是上述通信机制的基础。当前广泛使用的 RSA、Diffie–Hellman 和椭圆曲线密码分别依赖整数分解或离散对数问题。Shor 算法表明，具备足够规模的通用量子计算机可以在多项式时间内求解这两类问题[1]。对于需要长期保护的数据和生命周期较长的信息系统，密码迁移不能只在最后阶段替换一个算法名称，还要同步处理身份管理、协议组合、对象格式、运行资源和软件接入问题。

格密码以高维格上的计算问题为基础，是后量子公钥密码的重要技术路线。LWE、SIS 及其环、模和 NTRU 结构变体能够支持加密、签名、身份基加密和身份基签名，但它们并不是同一种实现的不同名称。矩阵与多项式表示、陷门形式、Gaussian 采样和参数规模的差异，会直接改变密钥尺寸、密文长度、运行时间和内存需求。理论上可组合的原语，未必能够在同一软件中直接交换。

因此，格上 IB-ME 的研究还包含明确的工程问题：怎样让多类 HIBE 与 IBS 遵守同一协议边界，怎样在不依赖后端私有矩阵的条件下完成组合，怎样承载超过 HIBE 明文容量的签名与应用消息，以及怎样降低结构化采样、对象编码和重复委派的成本。本作品以可替换的两层匿名 HIBE 与 IBS 通用构造为基础，通过多种真实后端、完整消息流程和针对性优化回答这些问题。第八章将结合代表方案进一步讨论其应用方式和部署条件。

1.2 国内外研究现状

身份基密码、格 HIBE、格 IBS 和 IB-ME 并非前后替代的单一路线。HIBE 与 IBS 分别沿加密和认证方向发展，IB-ME 研究则围绕双向身份匹配、匿名性和认证模型展开；Wang 等人的通用构造使这些支线在两层匿名 HIBE 与 IBS 的组合处汇合。图 1.2按照原始论文的发表时间和技术关系概括与本作品直接相关的节点，后文再说明各阶段解决的问题及留下的实现代价。

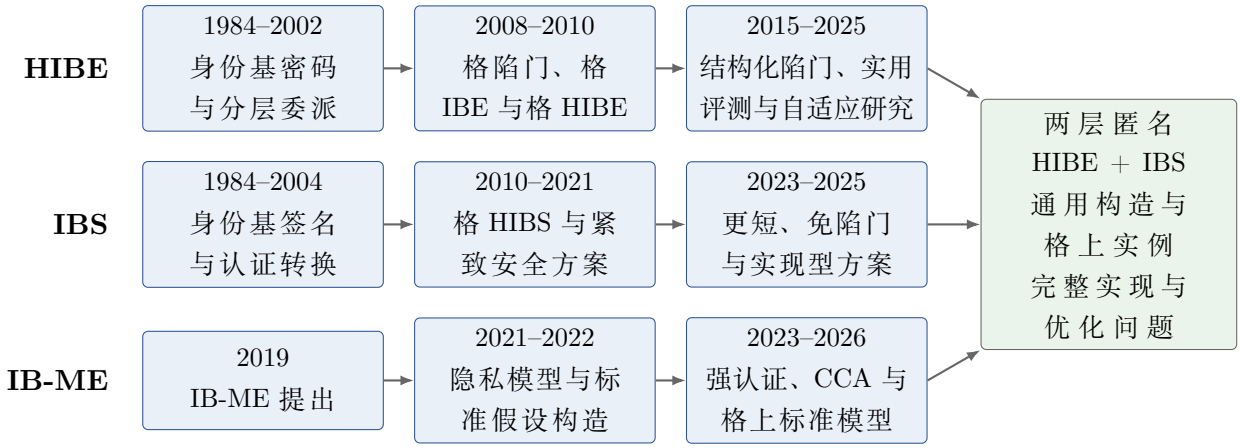


图 1.2 与本作品相关的三条研究支线及其汇合关系

1.2.1 身份基加密与分层委派的发展

Shamir 于 1984 年提出身份基密码的基本思想，但当时尚未给出完整实用的身份基加密方案 [2]。Boneh 和 Franklin 于 2001 年给出基于双线性对的 IBE 构造，使任意字符串作为加密目标成为可实现的密码功能[3]。Gentry 和 Silverberg 随后提出 HIBE，将单一发钥机构扩展为分层体系：上层持钥者可以为下级身份委派私钥，而下级不能反向获得父级或兄弟节点密钥 [4]。这一变化使身份基加密能够对应机构、部门、岗位和设备构成的管理层级。

格上身份基密码的关键基础来自陷门和短原像采样。GPV 工作在 2008 年给出格陷门、签名和匿名 IBE 构造[9]；Cash 等人的 Bonsai Trees 以及 Agrawal、Boneh 和 Boyen 的两项工作在 2010 年把格陷门委派用于 HIBE，分别解决非双线性对 HIBE、标准模型构造和委派后维度增长等问题 [10, 11, 12]。这些方案同时说明了格 HIBE 的主要代价：身份深度、陷门表示与委派方法会影响私钥、密文和计算规模，委派不能只被视为一个抽象接口。

后续研究继续沿结构化表示和安全模型推进。Lai 等人将 Micciancio–Peikert 型陷门扩展到理想格，以环元素代替部分普通矩阵，并用于具有身份匿名性的 (H) IBE[23]。LATTE 则面向公开安全通信场景完成 NTRU 结构 HIBE 的系统化实现与评测，表明 KeyGen、Delegate、Extract 和 Gaussian 采样是决定方案能否实际运行的核心环节[22]。2025 年的工作进一步推进了标准假设下自适应安全和匿名 HIBE 的理论构造[20]。理论能力不断增强的同时，具体方案在对象表示、委派成本和实现成熟度上的差异仍然显著，这也是本作品同时实现 LATTE 与 Lai 两条 HIBE 路线的研究背景。

1.2.2 身份基签名的格上实现路线

Shamir 在提出身份基密码时已经给出身份基签名（Identity-Based Signature，IBS）的设想。IBS 允许验证者使用签名者身份和系统公开参数验签，省去为每名用户单独传递公钥证书的步骤。Bellare、Namprempre 和 Neven 随后形式化了认证转换：发钥机构为用户生成的普通验签公钥签发身份证书，用户再用相应签名密钥签署消息[6]。这种路线能够复用成熟数字签名，但 IBS 签名需要携带用户公钥、机构证书和消息签名；Galindo 等人进一步研究了该转换对附加签名性质的适用范围 [27]。

格 IBS 的直接构造形成了另一条路线。Rückert 于 2010 年给出格上层次身份签名，展示了陷门委派与 GPV 型签名的组合[13]。Pan 和 Wagner 在 2021 年提出基于 SIS 与 Ring-SIS 的紧致安全 IBS，随后 Sageloli 等人简化参数和签名关系、去除一次昂贵的陷门委派，并给出具体参数 [26, 25]。这一路线使身份直接进入矩阵或环上的短向量关系，但陷门委派、预像采样和大尺寸用户密钥仍会带来明显的时间与空间成本。

近年的工作进一步分化出不同工程取舍。Luo 等人以两次 Lyubashevsky 型签名构造免陷门的模格 IBS，避免在发钥时进行格预像采样[35]；SRGG 将适合理想格 IBE 的预像采样与椭球 Gaussian 形式的 G + G 签名结合，并给出三组参数和概念验证实现[24]。Wang 等人还在格上 IB-ME 研究中改进 SIS 型 IBS，以缩短进入最终密文的签名[17]。因此，认证转换、矩阵 SIS、模格和环格方案虽然都提供 IBS 接口，其发钥方式、签名计算、对象尺寸和内存需求并不相同；这些差异正是通用组合设计必须保留而不能抹平的后端特征。

1.2.3 身份基匹配加密及格上构造的发展

Ateniese 等人在 CRYPTO 2019 提出匹配加密及其身份基特例，使发送端和接收端可以在不公开身份与策略的条件下进行双向匹配[14]。随后研究主要沿安全模型和构造基础两方面推进。Francati 等人在 2021 年研究无随机预言机的 IB-ME 并加强身份不匹配时的隐私描述 [15]；Chen 等人在 2022 年给出基于标准假设的构造[16]。这些工作仍主要依赖双线性群，因而尚未解决格上后量子实现问题。

Wang 等人把发送消息的认证范围和自我伪造问题纳入更强认证目标，并提出由自适应安全的两层匿名 HIBE 与 IBS 组成的通用构造。两层身份依次对应接收者和发送者，IBS 为发送者生成的协议对象提供认证；以格 HIBE 和格 IBS 实例化后，得到可抵抗量子攻击路线的 IB-ME 方案[17]。这一构造也是本作品公共协议的理论基础。其价值在于把 HIBE 和 IBS 的选择分离，但论文层面的可替换性并不自动给出统一的软件对象、传输格式或性能边界。

2023 年以来，相关研究又覆盖 CCA 隐私、标准格假设下的 IB-ME 以及标准模型中的格 IBS 与 IB-ME 框架[18, 19, 21]。研究重点已经从“能否实现双向身份匹配”扩展到不同隐私和认证目标、不同构造工具及后量子实例。与此相比，公开文献对多种目标参数 HIBE、IBS 进入同一完整消息软件后的接口适配、状态保存、载荷组成和同机比较讨论较少。由此产生的具体实现问题，构成下一节和本作品技术路线的出发点。

1.3 现有研究的工程实现问题

已有研究分别建立了分层身份加密、身份认证和双向身份匹配的理论工具，也给出了多种格上实例。但把这些原语组成一套能够发送应用消息、保存运行状态并替换后端的软件，并不是把论文中的算法接口依次连接即可。本作品选取的具体方案在对象结构、明文容量、采样方法和状态生命周期上存在明显差异，完整实现需要集中解决以下四类问题。

原语接口和对象表示不同。本作品采用的 HIBE 与 IBS 分别使用矩阵、多项式、模向量和 NTRU 对象，参数版本、身份映射、采样器和序列化方法也不相同。如果公共协议直接读取后端矩阵或陷门字段，每增加一种算法都需要修改协议代码，组合数量增加后也难以保证处理逻辑一致。

HIBE 明文容量与 IBS 签名尺寸不匹配。部分 HIBE 只适合加密几十字节，而 IBS 签名可能达到数 KiB、数百 KiB 甚至更大，应用消息也没有固定长度。将“签名 + 消息”直接作为 HIBE 明文，会使不少组合超过容量上限。完整系统需要分离短秘密封装和大载荷加密，同时防止两部分被替换或重组。

论文参数下的计算和存储开销较高。矩阵平方根、离散 Gaussian 采样和陷门委派若按稠密矩阵直接实现，可能产生远超常规工作站容量的中间对象。通用 JSON 表示虽然便于调试，却会重复保存大量整数的符号、分隔符和十进制字符。实现既要保持原有代数关系，也要减少不必要的矩阵展开和对象膨胀。

单次算法实验不能代表连续消息服务。系统建立、用户发钥和固定身份对的叶密钥委派可以被后续消息复用。如果每条消息都重新执行全部步骤，测试结果会被准备成本主导。软件需要区分长期状态、会话状态和逐消息对象，并分别测量首次、重复和恢复后的运行成本。

1.4 作品研究内容与技术路线

本作品首先定义 HIBE 与 IBS 相互独立的后端接口，并依据两层匿名 HIBE 与 IBS 的通用构造实现系统建立、发送者发钥、接收者发钥、消息加密和匹配解密五个公共算法。协议固定双方身份顺序和认证上下文，不读取后端内部的矩阵、陷门或采样状态。

其次，作品实现 LATTE、Lai 两种 HIBE，以及 BNN + ML-DSA、BNN + Falcon、Luo、SRGG、Sageloli 五种常规 IBS，形成十种端到端组合。另实现 Wang–PW 目标参数矩阵 SIS 后端，并完成其与 LATTE 的完整消息流程。本报告使用“Wang–PW”专指该具体 IBS 后端；Wang 等人提出的通用 IB-ME 构造则是第三章公共协议的理论基础。

再次，作品针对实际运行中最突出的三类瓶颈完成 Sageloli 结构化环采样、SRGG 范围感知编码和 Lai 接收会话。三项工作分别减少大型中间矩阵、序列化空间和固定身份对的重复委派成本。

最后，作品建设规范对象、状态持久化、命令行和浏览器操作入口，并记录系统建立、双方发钥、首次与重复收发、错误身份、密文篡改、对象尺寸和峰值内存。两种 HIBE 与五种常规 IBS 形成的十种组合还在同一机器、输入和线程配置下完成基本性能比较，用于区分 HIBE 与 IBS 对协议各阶段的影响。图 1.3给出从底层原语到应用入口的总体技术路线。

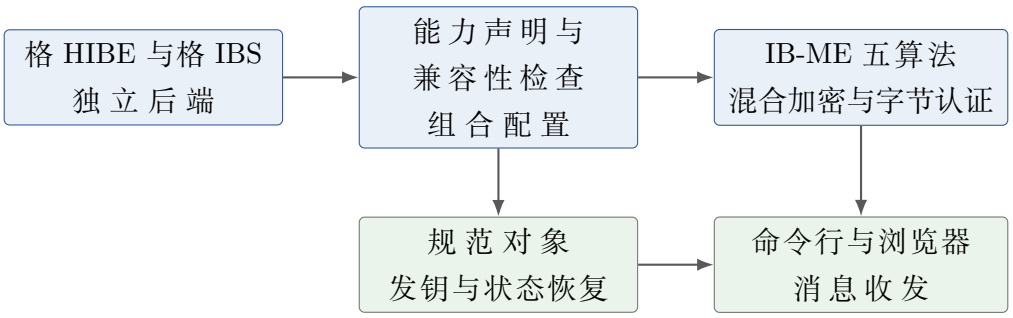


图 1.3 作品总体技术路线

1.5 主要工作

本作品围绕通用协议、具体算法和软件运行三个层次展开。协议层以两层匿名 HIBE 和 IBS 的通用构造为基础，统一双方身份顺序、五个公共算法和密文格式，并通过短种子混合加密解决 HIBE 明文容量与大尺寸签名之间的矛盾。算法层实现两种 HIBE 与五种常规 IBS 的十种组合，并接入 Wang–PW 这一目标参数矩阵 SIS 后端，使不同格结构可以在同一完整消息流程中运行。

作品进一步针对完整系统暴露出的主要成本改进实现方法。Sageloli 后端保留环结构，以逐频率低秩计算代替不可承受的稠密矩阵展开；SRGG 后端按照各字段的实际取值范围建立规范二进制编码；Lai 后端为固定身份对建立接收会话，复用已经生成的叶密钥。系统同时提供能力检查、固定发钥、状态保存与恢复、命令行和浏览器入口，并在统一条件下记录十种常规组合的时间、对象尺寸和运行内存。这些工作的创新性及其适用条件在第八章集中讨论。

1.6 报告结构

全文共九章。第 1 章说明研究背景、相关工作、工程问题和主要工作，第 2 章介绍格密码、身份基密码与 IB-ME 所需的基础知识；第 3 章给出通用协议、五算法和对象边界，第 4 章介绍已实现算法、组合矩阵及三组代表方案，第 5 章展开 Sageloli 采样、SRGG 编码和 Lai 接收会话三项优化；第 6 章说明软件架构、后端接入、状态恢复与操作流程，第 7 章给出十种组合及各项专项实验的结果，第 8 章归纳创新性并分析应用前景，第 9 章总结作品并提出后续工作。附录列出源码、数据、参数与图表的复现入口。

第二章　基础知识

本章建立理解后续设计所需的理论基础。内容从格与模运算开始，依次说明格困难问题、离散 Gaussian 与陷门、HIBE、IBS 以及 IB-ME，为第三章的通用协议和第四章的具体算法统一概念与符号。

2.1　数学符号与格基础

2.1.1　向量、矩阵与模运算

记整数集合为 $\mathbb{Z}$，模 q 整数环为 $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$。向量使用粗体小写字母表示，矩阵使用大写字母表示。对 $x \in \mathbb{Z}_q$，程序既可以保存区间 $[0, q)$ 中的非负代表元，也可以在需要计算范数时转换为中心区间 $(-q/2, q/2]$ 中的代表元；两种表示必须在接口处明确区分。

对向量 $\mathbf{x} = (x_1, \ldots, x_n)$，报告主要使用欧氏范数和无穷范数

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}, \qquad \|\mathbf{x}\|_\infty = \max_i |x_i|.$$

签名方案通常要求响应向量满足给定范数界；加密方案则要求误差足够小，使解密时能够区分编码区间。所有维数、模数和范数门都属于算法参数，不能由公共协议自行推断或修改。

表 2.1列出后文频繁使用的符号。

表 2.1　基础数学符号

符号	含义
n, m, d, k, ℓ	格维数、矩阵宽度、环次数或模块秩等整数参数，具体含义由方案给出
q	模数；$\mathbb{Z}_q$ 表示模 q 整数环
$A, \mathbf{a}$	公开矩阵或公开向量
$\mathbf{s}, \mathbf{e}$	秘密向量与小误差向量
$\|\cdot\|_2, \|\cdot\|_\infty$	欧氏范数与无穷范数
R_q	多项式商环 $\mathbb{Z}_q[x]/(f(x))$
$D_{\Lambda,s,\mathbf{c}}$	格 Λ 上中心为 $\mathbf{c}$、宽度为 s 的离散 Gaussian 分布

2.1.2　格与 q 元格

给定线性无关向量组成的基 $B = [\mathbf{b}_1, \ldots, \mathbf{b}_n] \in \mathbb{R}^{m \times n}$，由 B 生成的格为

$$\Lambda(B) = \{B\mathbf{z} : \mathbf{z} \in \mathbb{Z}^n\}.$$

同一个格可以具有许多不同的基。长而近似相关的公开基便于描述格，短而接近正交的基则可以作为陷门，帮助持有者求短向量或短原像。格密码利用“公开描述容易生成、没有陷门时相关计算困难”这一差异构造加密和签名。

模 q 矩阵 $A \in \mathbb{Z}_q^{n\times m}$ 可以定义两类常见 q 元格：

$$\Lambda_q^{\perp}(A) = \{\mathbf{x} \in \mathbb{Z}^m : A\mathbf{x} = \mathbf{0} \pmod q\},$$

以及给定目标 $\mathbf{u}$ 的陪集

$$\Lambda_q^{\mathbf{u}}(A) = \{\mathbf{x} \in \mathbb{Z}^m : A\mathbf{x} = \mathbf{u} \pmod q\}.$$

身份基发钥和格签名经常需要在后一集合中找到满足范数要求的短向量。主陷门的作用不是改变目标方程，而是使合法发钥机构能够高效完成这种短原像采样。

2.2 格上困难问题及结构变体

2.2.1 LWE 问题

学习带误差问题（Learning With Errors，LWE）由 Regev 系统提出并用于构造公钥密码[8]。取均匀矩阵 $A \in \mathbb{Z}_q^{m\times n}$、秘密 $\mathbf{s} \in \mathbb{Z}_q^n$ 和小误差 $\mathbf{e} \in \mathbb{Z}^m$，观察值为

$$\mathbf{b} = A\mathbf{s} + \mathbf{e} \pmod q.$$

搜索型 LWE 要求从 $(A, \mathbf{b})$ 恢复 $\mathbf{s}$；判定型 LWE 要求区分 $(A, \mathbf{b})$ 与均匀样本。若没有误差，线性方程可以直接求解；小误差使恢复秘密成为格上的困难问题。格加密通常将明文或封装种子叠加在这类带误差关系上，合法密钥可以消去主线性项并解码小扰动。

2.2.2 SIS 问题

短整数解问题（Short Integer Solution，SIS）与 Ajtai 建立的平均情形格问题有关[7]。给定均匀矩阵 $A \in \mathbb{Z}_q^{n\times m}$，目标是找到非零短向量 $\mathbf{z}$，满足

$$A\mathbf{z} = \mathbf{0} \pmod q, \qquad 0 < \|\mathbf{z}\|_2 \le \beta.$$

非齐次版本给出目标 $\mathbf{u}$，要求寻找短向量 $\mathbf{z}$ 使 $A\mathbf{z} = \mathbf{u} \pmod q$。格签名常把身份和消息哈希为目标向量，再由陷门采样满足方程的短原像。验证者检查模方程和范数，而不知道陷门的参与者难以为新目标构造合格短向量。

2.2.3 环格、模格与 NTRU 结构

普通矩阵格直接保存 A 及其向量，表达灵活，但公开矩阵和密钥可能较大。环格把对象组织为商环

$$R_q = \mathbb{Z}_q[x]/(x^N + 1)$$

中的多项式，环乘法对应具有特定结构的线性变换。Ring-LWE 和 Ring-SIS 由此获得紧凑表示和快速多项式运算[30]。

模格在 R_q 上使用低秩向量或矩阵，在普通矩阵格的灵活性和环格的结构性之间折中；对应问题通常称为 Module-LWE 与 Module-SIS[31]。ML-DSA 和本作品采用的多种原生 IBS 属于这一技术路线。

NTRU 结构同样在多项式环上运算，通过短多项式关系形成结构化格。该结构可以用于加密、签名和陷门委派。LATTE 利用 NTRU 结构完成 HIBE，Falcon 利用 NTRU 格完成普通数字签名。这里的“普通矩阵格、环格、模格和 NTRU 结构格”描述对象结构与困难问题来源，不表示彼此之间的安全等级高低。

表 2.2 本作品涉及的格结构及其一般工程特征

结构	典型对象	一般工程特征
普通矩阵格	$\mathbb{Z}_q$ 上的矩阵与整数向量	表达直接，目标参数下矩阵和陷门对象可能较大
环格	商环多项式与环向量	可利用卷积和频域结构，需处理环采样与数值变换
模格	小秩环向量或环矩阵	在结构复用与方案灵活性之间折中
NTRU 结构格	满足短多项式关系的结构化基	对象较紧凑，可支持快速签名、加密和陷门委派

2.3 离散 Gaussian 与格陷门

2.3.1 离散 Gaussian 采样

对格 Λ、中心 $\mathbf{c}$ 和宽度 $s > 0$，连续 Gaussian 权重写为

$$\rho_{s,\mathbf{c}}(\mathbf{x}) = \exp\left(-\pi \frac{\|\mathbf{x}-\mathbf{c}\|_2^2}{s^2}\right).$$

离散 Gaussian 分布 $D_{\Lambda,s,\mathbf{c}}$ 在 Λ 上按归一化后的 $\rho_{s,\mathbf{c}}$ 取样。当采样空间是 q 元格的某个陪集时，输出同时满足目标模方程和短向量要求。宽度过小可能无法满足分布条件，宽度过大又会增大签名范数或解密误差，因此必须采用方案给出的参数和采样方法。

实际实现中，普通矩阵格、环格和 NTRU 结构使用的采样器并不相同。采样过程可能包含扰动采样、Gram–Schmidt 信息、FFT / NTT 变换、Babai 反演或逐系数拒绝采样。均匀随机数不能替代离散 Gaussian；数值近似也必须在输出后接受精确模方程和范数检查。

2.3.2 陷门生成、委派与短原像采样

格陷门是一份能够帮助合法持有者完成短原像采样的秘密辅助信息。典型算法可以概括为：

1. TrapGen 生成统计上接近均匀的公开矩阵 A 及短陷门 T_A；
2. SamplePre$(A, T_A, \mathbf{u})$ 为目标 $\mathbf{u}$ 采样短向量 $\mathbf{x}$，使 $A\mathbf{x} = \mathbf{u} \pmod q$；

3. 委派算法把已有陷门扩展到包含新身份块的公开矩阵，同时保持可用的短基或结构化陷门。

Micciancio–Peikert 陷门框架使用公开 gadget 矩阵组织这些关系[29]。HIBE 通过陷门委派沿身份层级生成密钥，原生格 IBS 通过短原像采样生成身份密钥或签名。后文的 Sageloli、Lai 和 Wang–PW 虽采用不同结构，其核心计算都可以在“公开目标关系 + 秘密短陷门 + 短原像采样”这一框架下理解。

2.4 身份基加密与分层身份基加密

2.4.1 身份基加密

身份基加密（Identity-Based Encryption，IBE）使用身份字符串代替传统用户公钥。可信发钥机构运行 Setup 得到主公钥 mpk 和主秘密 msk；用户通过 $\mathsf{Extract}(\mathsf{msk}, \mathsf{id})$ 领取身份私钥；发送者以 id 和 mpk 加密；接收者使用对应身份私钥解密。其正确性要求对任意合法身份和消息，诚实生成的身份私钥能够恢复消息。

IBE 把公钥发现问题转换为身份规范与发钥机构管理问题。通信者只需知道对方规范身份和系统公开参数，但发钥机构必须可靠核验身份、保护主秘密并管理密钥生命周期。因而，IBE 适合具有明确组织边界和统一身份管理的系统。

2.4.2 HIBE 的分层身份与密钥委派

分层身份基加密（Hierarchical Identity-Based Encryption，HIBE）把身份扩展为路径 $(\mathsf{id}_1, \ldots, \mathsf{id}_d)$。除 Setup、$\mathsf{Extract}$、$\mathsf{Encrypt}$ 和 $\mathsf{Decrypt}$ 外，HIBE 还包含密钥委派 Derive：持有前缀路径密钥的实体可以为下一层身份生成子密钥，无需再次调用根主秘密。

本项目使用两层 HIBE。第一层是接收身份 r，第二层是发送身份 s：

$$(\mathsf{id}_1, \mathsf{id}_2) = (r, s).$$

发钥机构为 r 生成一级密钥 dk_r，接收者再根据认可的发送身份 s 委派叶密钥 $\mathsf{dk}_{r,s}$。发送者以同一身份路径加密。路径中任一身份改变，生成的叶密钥都不能按正确路径恢复密文。

除消息机密性外，IB-ME 还需要 HIBE 具备身份匿名性。匿名性要求攻击者不能仅从密文判断其对应的身份路径。该性质用于避免公开密文直接泄露“谁向谁发送”的关系。具体安全模型与参数由各 HIBE 方案给出，公共协议只调用其规定的算法接口。

2.5 身份基签名

一套身份基签名包含四个核心算法：

$$\begin{aligned}(\mathsf{ipk}, \mathsf{imsk}) &\leftarrow \mathsf{Setup}(1^\lambda),\\ \mathsf{sk}_{\mathsf{id}} &\leftarrow \mathsf{KeyExtract}(\mathsf{imsk}, \mathsf{id}),\\ \sigma &\leftarrow \mathsf{Sign}(\mathsf{sk}_{\mathsf{id}}, M),\\ b &\leftarrow \mathsf{Verify}(\mathsf{ipk}, \mathsf{id}, M, \sigma).\end{aligned}$$

正确性要求合法身份密钥产生的签名能够通过验证。不可伪造性要求攻击者即使获得其他身份的密钥和多条签名，也不能为未获授权的身份与新消息生成有效签名。

IBS 可以采用原生身份关系，也可以由普通数字签名经认证转换得到。认证转换中，发钥机构为用户普通验签公钥签发与身份绑定的证书，完整 IBS 签名携带用户公钥、机构证书和用户签名；验证者先检查证书，再检查用户签名。原生格 IBS 则让身份直接进入目标向量、挑战或陷门采样关系。两类方案在接口上都完成“身份 + 消息 + 签名”的公开验证，但密钥尺寸、签名尺寸和运行成本不同。

在 IB-ME 中，IBS 的职责是认证发送者。接收端使用自己认可的发送身份验签，因而更换 s 不仅会改变 HIBE 第二层路径，也会改变 IBS 验证输入。两种检查共同把发送身份绑定到消息恢复过程。

2.6 身份基匹配加密与通用构造

IB-ME 包含系统建立、发送者发钥、接收者发钥、消息加密和匹配解密五项算法。发送者发钥为身份 s 生成加密所需的发送密钥，接收者发钥为身份 r 生成接收密钥。加密算法同时接收发送密钥、接收身份 r 和消息 M；解密算法同时接收接收密钥、认可发送身份 s' 和密文。正确性条件为 $r = r'$ 且 $s = s'$ 时恢复 M。

Wang 等人的通用构造使用两层匿名 HIBE 与 IBS 实现这一语法[17]。HIBE 沿 (r, s) 路径保护内部秘密，IBS 以 s 认证发送者生成的协议消息。两种原语承担不同职责：HIBE 决定哪一条双方身份路径能够恢复秘密，IBS 决定恢复后的消息是否确由指定发送身份产生。通用构造不要求二者使用相同的维度、模数或格结构。

该构造讨论的基本目标包括消息与身份隐私、发送来源认证以及合法用户之间的自我伪造防护。软件实现还需要处理完整密文字节认证、规范解析和状态恢复，但这些工程机制不替代底层 HIBE 与 IBS 的密码性质。第三章将说明本作品如何在不读取原语私有对象的前提下完成五算法与完整传输格式。

2.7 本章小结

本章从格、模运算和短向量出发，介绍了 LWE、SIS 及其环、模和 NTRU 结构变体，并说明离散 Gaussian 与格陷门如何支持短原像采样。随后给出 IBE、HIBE、IBS 和 IB-ME 的基本语法及相互关系。这些内容构成后续通用设计的理论基础；具体采用的算法、参数和代表组合将在第四章集中介绍。

第三章　通用 IB-ME 系统设计

本章只讨论所有算法组合共享的系统设计。底层 HIBE 和 IBS 被视为相互独立的密码后端，具体方案、参数和代表组合留到第四章介绍。公共协议负责双方身份顺序、短种子封装、发送身份认证、完整密文字节保护、规范对象和异常处理。

3.1　设计目标与总体架构

通用设计需要满足四项要求。第一，不同 HIBE 与 IBS 能够独立替换，公共协议不读取后端内部的矩阵、多项式和陷门。第二，HIBE 只处理固定长度短秘密，使签名尺寸和应用消息长度不受其明文容量直接限制。第三，发送身份、接收身份、应用消息和完整密文字节保持不可拆分的绑定。第四，公开对象、角色密钥和密文能够规范保存，并在程序重启后恢复。

图 3.1给出公共协议的层次。原语层实现 HIBE 和 IBS；适配层统一算法入口、对象编码和能力声明；协议层执行 IB-ME 五算法；状态层负责固定发钥和对象恢复；命令行与浏览器仅调用同一应用服务，不各自实现密码流程。

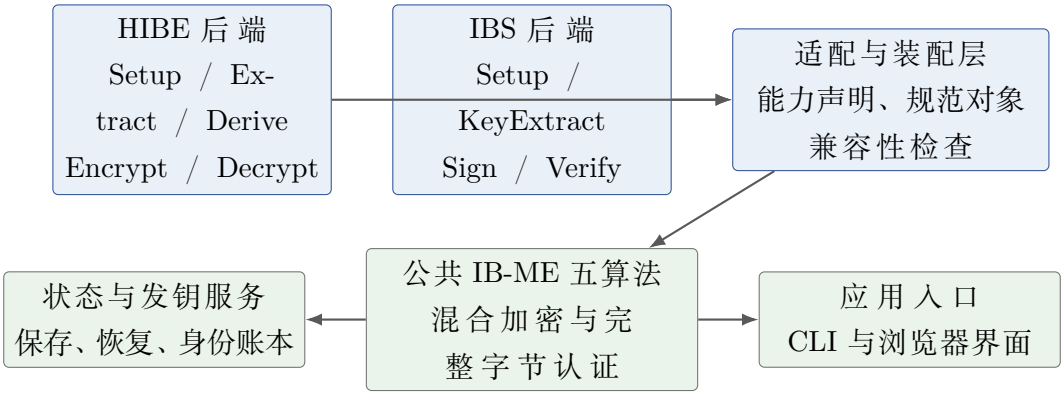


图 3.1　通用 IB-ME 系统的分层架构

3.2　参与者、对象与身份顺序

系统包含可信发钥机构、发送者和接收者。发钥机构保管 HIBE 与 IBS 的主秘密，公开两套公开参数。发送者领取与自身身份 s 绑定的签名密钥，并在加密时指定接收身份 r；接收者领取身份 r' 的 HIBE 一级密钥，并在解密时填写认可发送身份 s'。

HIBE 路径固定为 (r, s)，即第一层为接收者，第二层为发送者。接收者持有一级密钥 $\mathsf{dk}_{r'}$，再根据 s' 派生叶密钥 $\mathsf{dk}_{r',s'}$。这一顺序贯穿对象编码、状态文件、签名输入和错误身份测试，后端不能自行交换两层身份。

表 3.1列出公共协议符号。HIBE 与 IBS 分别维护自己的参数和对象；公共层只保存严格类型对象及其规范字节，不要求二者共享 q、维度或环。

表 3.1 通用协议主要符号

符号	含义
$r, s;\ r', s'$	密文指定的接收者与实际发送者；接收密钥身份与接收者认可的发送者
$\mathcal{H}, \mathcal{S}$	两层匿名 HIBE 后端与 IBS 后端
$\mathsf{hpk}, \mathsf{hmsk}$	HIBE 公开参数与主秘密
$\mathsf{ipk}, \mathsf{imsk}$	IBS 公开参数与主秘密
$\mathsf{dk}_r, \mathsf{sk}_s$	接收者一级密钥与发送者签名密钥
M, σ	应用消息与 IBS 签名
z, K	32 字节随机种子与由其派生的对称密钥
$\mathsf{ovk}, \mathsf{osk}, \tau$	一次性验签公钥、一次性签名私钥与外层签名
$\mathrm{LP}(x)$	4 字节长度前缀与字节串 x 的拼接

系统公开对象 MPK 包含 hpk、ipk 和组合配置 suite；主状态 MSK 保存 hmsk、imsk 及发钥记录。发送密钥 EK_s 绑定身份 s 和 IBS 用户密钥，接收密钥 DK_r 绑定身份 r 和 HIBE 一级密钥。所有对象都记录同一组合标识、参数摘要和编码版本。

3.3 原语接口与兼容性检查

HIBE 后端提供 Setup、Extract、Derive、Encrypt 和 Decrypt；IBS 后端提供 Setup、KeyExtract、Sign 和 Verify。每个后端还要提供公开对象、秘密对象和密文 / 签名对象的规范编码与加载方法。

能力声明描述身份字节上限、HIBE 明文长度、IBS 签名输入上限、签名长度上界、对象编码版本以及是否支持持久化。装配层在 Setup 前完成以下检查：

1. 两层 HIBE 能够接收规范化后的接收身份与发送身份；
2. HIBE 能够加密固定的 32 字节种子；
3. IBS 能够签署包含接收身份、应用消息和一次性公钥承诺的规范字节；
4. 对称载荷预算能够容纳 IBS 签名、消息和格式开销；
5. 两个后端的对象版本、随机源要求和保存能力满足组合配置。

检查结果给出该组合的应用消息上限。这样，不兼容配置在生成主密钥之前即被拒绝；公共协议无需通过运行失败猜测后端容量。

3.4 IB-ME 五算法伪代码

伪代码用于明确输入、输出、调用顺序和拒绝条件。它描述公共协议，不展开任何具体格后端的矩阵运算；第四章再说明各项接口如何由具体算法实现。

算法 3.1 系统建立 Setup

输入： 组合配置 suite，密码随机源。

输出： 系统公开对象 MPK，主状态 MSK。

1. 读取 HIBE 与 IBS 能力声明，执行身份、容量、编码和持久化兼容性检查；
2. 运行 $(\mathsf{hpk}, \mathsf{hmsk}) \leftarrow \mathcal{H}.\mathsf{Setup}()$；
3. 运行 $(\mathsf{ipk}, \mathsf{imsk}) \leftarrow \mathcal{S}.\mathsf{Setup}()$；
4. 计算公开参数、参数快照和 suite 的规范摘要；
5. 输出 $\mathsf{MPK} = (\mathsf{hpk}, \mathsf{ipk}, \mathsf{suite})$ 及仅由机构保存的 $\mathsf{MSK} = (\mathsf{hmsk}, \mathsf{imsk}, \mathsf{suite})$。

算法 3.2 发送者密钥生成 SenderKeyGen

输入： MSK，发送身份 s。

输出： 发送密钥 EK_s 或拒绝 $\perp$。

1. 按发送者域标识和长度前缀规范编码 s，检查身份长度；
2. 调用 $\mathsf{sk}_s \leftarrow \mathcal{S}.\mathsf{KeyExtract}(\mathsf{imsk}, s)$；
3. 检查返回对象类型、参数摘要和身份绑定；
4. 输出 $\mathsf{EK}_s = (s, \mathsf{sk}_s, \mathsf{suite})$，并由固定发钥服务记录摘要。

算法 3.3 接收者密钥生成 ReceiverKeyGen

输入： MSK，接收身份 r。

输出： 接收密钥 DK_r 或拒绝 $\perp$。

1. 按接收者域标识和长度前缀规范编码 r，检查身份长度；
2. 调用 $\mathsf{dk}_r \leftarrow \mathcal{H}.\mathsf{Extract}(\mathsf{hmsk}, r)$；
3. 检查一级密钥的参数摘要、层级和身份绑定；
4. 输出 $\mathsf{DK}_r = (r, \mathsf{dk}_r, \mathsf{suite})$。

算法 3.4　消息加密 Encrypt

输入： MPK，EK_s，接收身份 r，消息 M。
输出： 规范密文 C 或拒绝 $\perp$。

1. 检查组合摘要、身份和消息上限，生成一次性密钥对 $(\mathsf{ovk}, \mathsf{osk})$；
2. 计算 $d_{\mathrm{ovk}} = H_{\mathrm{commit}}(\mathsf{ovk})$，将 M 和 d_{ovk} 编码为内层信封 E，再令 $E_{\mathrm{sig}} = \mathrm{EncMsg}(r, E)$；
3. 生成 $\sigma \leftarrow \mathcal{S}.\mathsf{Sign}(\mathsf{sk}_s, E_{\mathrm{sig}})$，编码内部载荷 P；
4. 新取 32 字节种子 z，调用 $C_0 \leftarrow \mathcal{H}.\mathsf{Encrypt}(\mathsf{hpk}, (r, s), z)$；
5. 由 z 派生 K，使用 AES-256-GCM 加密 P 得到 C_1；
6. 对公开上下文、ovk、C_0 和 C_1 的规范字节生成一次性签名 τ；
7. 输出 $C = \mathrm{Encode}(\mathsf{ctx}, \mathsf{ovk}, C_0, C_1, \tau)$。

算法 3.5　匹配解密 Decrypt

输入： MPK，$\mathsf{DK}_{r'}$，认可发送身份 s'，密文 C。
输出： 消息 M 或拒绝 $\perp$。

1. 规范解析 C，检查版本、字段长度、配置摘要和公开上下文；
2. 用 ovk 验证 τ；失败则返回 $\perp$；
3. 计算 $\mathsf{dk}_{r',s'} \leftarrow \mathcal{H}.\mathsf{Derive}(\mathsf{dk}_{r'}, s')$；
4. 解开 C_0 得到 z，派生 K 并验证 GCM 标签，恢复 P；
5. 解析 E 与 σ，计算 $E_{\mathrm{sig}} = \mathrm{EncMsg}(r', E)$，以身份 s' 调用 $\mathcal{S}.\mathsf{Verify}$；
6. 解析 E 中的 d_{ovk} 与 M，核对 d_{ovk} 来自外层 ovk；
7. 所有检查通过后返回 M；任一步失败均返回 $\perp$。

3.5　基于短种子封装的混合加密

IBS 签名尺寸从数 KiB 到数 MiB 不等，而 HIBE 直接明文容量通常只有几十字节。公共协议采用 KEM–DEM 式混合结构：HIBE 只加密固定的 32 字节随机种子，对称加密承担签名和应用消息组成的可变长度载荷。

外部身份先用角色标识、长度前缀和零填充编码为定长输入。当前 64 字节外部身份对应 86 字节内部身份；接收者与发送者使用不同角色字节。HIBE 封装和密钥派生为

$$
\begin{aligned}
C_0 &\leftarrow \mathcal{H}.\mathsf{Encrypt}(\mathsf{hpk}, (\mathrm{Pad}_P(r), \mathrm{Pad}_C(s)), z),\\
K &\leftarrow \text{HKDF-SHA256}(z; \mathsf{ctx}_H, \mathsf{D}_{\mathrm{kdf}}),\\
C_1 &\leftarrow \text{AES-256-GCM.}\,\mathrm{Enc}(K, n, P; \mathsf{AAD}).
\end{aligned}
$$

这里 n 是每条消息新生成的 12 字节 nonce。HKDF 和 GCM 分别采用 RFC 5869 与 NIST SP 800-38D 规定的原语[32, 33]。附加认证数据包含组合上下文、HIBE 胶囊 C_0、nonce 和载荷长度。修改胶囊或把对称密文移动到另一组合中，都会导致认证失败。

混合结构把底层 HIBE 需要处理的明文固定为 32 字节。应用容量转而由 IBS 签名输入上限、签名最大尺寸和对称载荷预算共同决定，装配层可以在建立主密钥之前计算消息上限。

3.6 发送身份绑定与完整密文字节认证

仅验证解密后的 IBS 签名不能覆盖传输中的版本、字段边界和 HIBE 胶囊。本作品使用 Lamport 型一次性签名连接内外两层认证[34]。加密时先把一次性验签公钥承诺写入 IBS 签名消息：

$$
\begin{aligned}
d_{\mathrm{ovk}} &= H_{\mathrm{commit}}(\mathsf{ovk}),\\
E &= \mathsf{D}_{\mathrm{bound}} \parallel \mathrm{LP}(d_{\mathrm{ovk}}) \parallel \mathrm{LP}(M),\\
E_{\mathrm{sig}} &= \mathrm{EncMsg}(r, E),\\
\sigma &\leftarrow \mathcal{S}.\mathsf{Sign}(\mathsf{sk}_s, E_{\mathrm{sig}}),\\
P &= \mathsf{D}_{\mathrm{sig}} \parallel \mathrm{LP}(\sigma) \parallel \mathrm{LP}(E).
\end{aligned}
$$

得到 HIBE 胶囊和对称密文后，再对全部内层传输字节签名：

$$
\tau \leftarrow \mathsf{OTS.Sign}_{\mathsf{osk}}(\mathsf{D}_{\mathrm{ct}} \parallel \mathrm{LP}(\mathsf{ctx}) \parallel \mathrm{LP}(\mathsf{ovk}) \parallel \mathrm{LP}(C_0, C_1)).
$$

接收端先验证外层一次性签名，再解密载荷并检查 IBS 签名中的 d_{ovk}。攻击者即使重新生成一次性密钥并为修改后的外层字节签名，也不能让原 IBS 签名承诺新的公钥；原一次性签名也不能验证任何变化后的胶囊或对称密文。

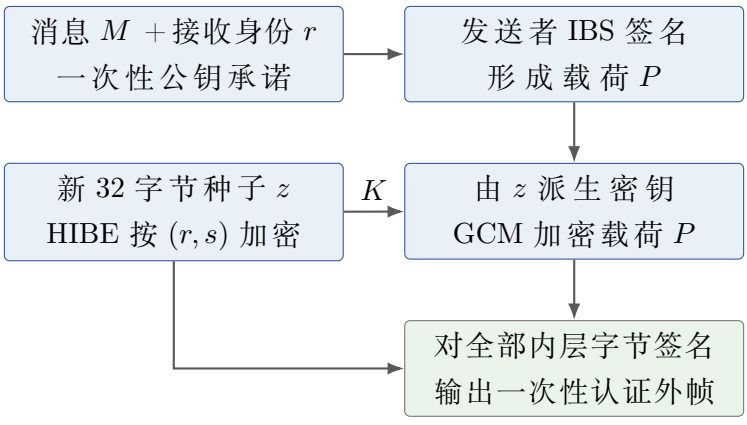


图 3.2 身份签名、短种子封装与完整字节认证的关系

3.7 规范编码与状态边界

所有公开对象、主状态、角色密钥和密文均使用类型标识、版本、参数摘要和长度前缀。编码器固定字段顺序，解码器检查总长度、字段上限、未知或重复字段、整数范围和末尾未使用位。解码后重新编码必须得到相同字节，从而避免同一数学对象存在多种可接受表示。

长期状态包括主秘密、已发身份记录、发送者密钥和接收者一级密钥；会话状态只保存某次固定身份对的叶密钥和上下文；逐消息对象包括随机种子、GCM nonce、一次性签名密钥和密文。三类状态采用不同生命周期，逐消息随机量不得写入固定发钥记录，会话叶密钥也不作为长期角色密钥导出。

恢复对象时，程序先加载公开配置，再检查对象中的算法名、参数摘要和身份绑定。恢复过程不重新发钥，也不改变既有身份对象。这样，命令行和浏览器入口可以在不同进程中使用同一套已生成状态。

3.8 正确性与异常处理

当 $r = r'$ 且 $s = s'$ 时，接收端派生的叶密钥与加密路径一致，HIBE 恢复同一个 z；HKDF 得到同一 K，GCM 恢复载荷 P；IBS 以同一发送身份验证签名，内外一次性公钥承诺也一致，最终输出 M。

接收身份错误时，一级密钥不能形成正确叶密钥；认可发送身份错误时，HIBE 第二层路径和 IBS 验签身份同时改变。应用消息、IBS 签名或一次性公钥承诺变化时，IBS 验证失败；胶囊、对称密文、上下文、nonce 或外层签名变化时，一次性签名或 GCM 验证失败。

解码器先检查魔数、版本、总长度和字段上限，再分配大型对象。未知字段、重复字段、截断、追加、非规范整数、错误对象类型和配置摘要不匹配均直接拒绝。运行环境缺少依赖或超过资源上限时，程序返回运行错误，不把系统故障解释为身份错配。

3.9 本章小结

本章给出了与具体格方案无关的 IB-ME 公共协议。五算法伪代码明确了原语调用和拒绝顺序；短种子封装解决 HIBE 明文容量与大签名之间的矛盾；IBS 和一次性签名分别绑定发送身份与完整密文字节；规范对象和状态边界支持固定发钥与重启恢复。由此，不同后端可以共享同一身份语义、处理顺序和失败结果，而不需要公共协议读取其内部矩阵、环元素或陷门。

第四章　算法实现与代表方案

本章介绍已经接入通用协议的具体密码算法，并说明三组代表方案的选择依据。两种 HIBE 和五种常规 IBS 形成十种端到端组合；Wang–PW 作为第六种 IBS 后端，已与 LATTE 完成一套目标参数组合。各算法保持独立参数和对象格式，公共协议仅通过第三章定义的接口调用。

4.1　分层身份基加密实现

4.1.1　LATTE 两层身份委派机制

LATTE 在 NTRU 结构中实现量子安全 HIBE[22]。作品采用 LATTE-3 参数：多项式次数 $N = 1024$、模数 $q = 68718428161$、分层深度为 2。主陷门由短 NTRU 关系构成，接收身份 r 和发送身份 s 分别映射为两层身份多项式。发钥机构为 r 生成一级密钥，接收端再为 s 委派叶密钥。

加密端按 (r, s) 生成 Ring-LWE 型胶囊，叶密钥消去公开环项后恢复 32 字节随机种子。实现随后进行确定性重加密核对，排除错误结果落入解码区间的情况。公开参数、主陷门、一级密钥、叶密钥和密文采用不同对象类型，参数快照与对象一同保存。

实现复用 LATTE 作者代码中的 Gaussian 采样和 ffSampling 路径，并将原生对象封装为严格的 Python 后端。每次原生采样后，适配层使用精确环算术检查 NTRU 关系、委派关系和对象形状。LATTE 建立、一级发钥和在线解密成本较低，因此 A、B 两组代表都选用该 HIBE，使两组主要差别集中在 IBS。

4.1.2　Lai 分层身份块扩展机制

Lai 方案利用理想格陷门完成分层委派[23]。根公开向量与短陷门满足

$$\mathbf{a}_0^{\mathsf{T}} R + \mathbf{a}_1 = \mathbf{g}.$$

两层身份分别映射为二进制环元素 id_1 和 id_2，完整身份公开项为

$$\mathbf{a}_{\mathrm{ID}} = \mathbf{u}_0 + \mathrm{id}_1 \mathbf{u}_1 + \mathrm{id}_2 \mathbf{u}_2.$$

每委派一层，后端向公开矩阵追加身份块，并为扩展后的矩阵生成新陷门。接收身份对应第一层，认可发送身份对应第二层。

C 组采用 $n = 2048$、$q = 274877976577$、$k = 39$、深度 2、$p = 2$ 的 C2 配置。身份映射使用二进制系数，避免身份乘短矩阵时放大噪声；叶陷门通过 Babai 反演恢复种子，并检查误差的模 p 结构。该后端的主要成本是二级委派，因此第五章进一步引入接收会话，使固定身份对只生成一次叶密钥。

4.2 身份基签名实现

4.2.1 BNN 认证转换与 ML-DSA / Falcon 接入

认证转换以普通数字签名为基础构造 IBS[27]。系统建立时，发钥机构生成机构签名密钥；为身份 s 发钥时，先生成用户普通签名密钥对 $(\mathsf{vk}_s, \mathsf{xk}_s)$，再由机构签署身份与用户验签公钥的规范编码：

$$\mathsf{cert}_s = \mathsf{DS.Sign}_{\mathsf{sk}_{\mathrm{CA}}}(\mathsf{Enc}(s, \mathsf{vk}_s)).$$

完整 IBS 签名携带用户公钥、机构证书和用户签名。验证端先检查证书，再检查用户签名。

同一转换核心接入 ML-DSA 与 Falcon。ML-DSA 实现采用 FIPS 204 对象格式[28]；A 组选用 ML-DSA-44，其普通公钥为 1,312 字节、普通签名为 2,420 字节，转换后的 IBS 签名为 6,237 字节。Falcon 后端采用 Falcon-512 和 Falcon-1024 的固定作者实现及 padded 格式[36]。两者共享机构证书流程，但底层密钥、签名编码和版本号相互独立。

4.2.2 Luo 模格身份签名

Luo 后端实现原生模格 IBS[35]。身份哈希、消息挑战、响应和提示信息直接进入模格签名关系，验证者使用公开参数、身份和消息完成验签。后端同时保留参考表示和无损紧凑表示，两种格式加载后恢复为同一数学对象，签名与验签算法不变。

Luo 分别与 LATTE、Lai 完成组合。它不属于 A、B、C 三组重点代表，但用于检验公共协议能否接入另一类原生模格 IBS，也为十种常规组合提供独立于 SRGG 的模格路线。

4.2.3 SRGG 身份目标与消息挑战关系

SRGG 在环 $R_Q = \mathbb{Z}_Q[X]/(X^n + 1)$ 上采用模块格结构。公开参数记为 P，其中的公开矩阵写为

$$A = [I_d \mid \widehat{A} \mid B].$$

身份 s 决定目标 $h = H_1(P, s)$，用户短见证 y 满足 $Ay = h \pmod{Q}$。令 $\zeta = 1 + X^{n/2}$，$\zeta^* = 1 - X^{n/2}$ 为其环伴随，并令 $j = \zeta^* e_1$，其中 e_1 是首个单位向量。签名时，p_0 是与主陷门相关、维数与响应一致的扰动向量，$c \in \{0, 1\}^n$ 是由 H_2 生成的二元挑战多项式，$u \in R_Q$ 按方案从以 $-c/\zeta$ 为中心的标量 Gaussian 中抽取。签名构造矩阵和响应为

$$M_h = [\,2A \mid Qj - 2h\,] \pmod{2Q}, \qquad z = p_0 + \begin{bmatrix} y \\ 1 \end{bmatrix} (\zeta u + c),$$

验证者由 $W' = M_h z - Qjc$ 恢复承诺 W'，重新计算挑战并检查响应范数[24]。这些量均在 R_Q 或相应环模中运算；p_0 负责引入相关扰动，y 把身份目标带入响应，$\zeta u + c$ 则把本次消息挑战绑定到签名。

B 组采用 PARAM-I：多项式次数 $n = 512$、模块秩 2、$p = 16383$、$Q = 15744063$，参数评估中的量子密钥恢复与伪造成本分别约为 95 bit 和 93 bit。该后端包含大量不同取值范围的整数系数，参考 JSON 对象较大。第五章的范围感知编码为公钥、主状态、用户密钥和签名分别选择位宽，在不改变数学对象的前提下减少保存与传输字节。

4.2.4 Sageloli 环陷门与非齐次签名方程

Sageloli 的公开向量与主陷门 R 满足

$$A = [\bar{A} \mid g - \bar{A}R].$$

为身份 s 发钥时，算法生成身份盐 ρ 并采样身份陷门 T_s，使

$$AT_s = H_1(P, \rho, s) - g \pmod{q}.$$

签名另取消息盐 η，通过相关扰动和三进制陪集采样得到响应 z；验证端检查范数和非齐次模方程[25]。

C 组选用 PARAM-I：$(d, k, \ell) = (2048, 65, 132)$、$q = 3^{65}$、$r^2 = 49$。PARAM-II 保持同一算法并增大参数。论文参数下若把环乘法展开为普通矩阵，采样中间对象无法在常规工作站内保存。第五章说明如何利用环结构和逐频率低秩计算完成两档参数的发钥与签名。

4.2.5 Wang–PW 目标参数矩阵 SIS 后端

Wang–PW 由 Wang 基本 IBS 与 Pan–Wagner 自适应转换组成，采用矩阵 SIS 关系，并分别为身份和消息加入随机盐[17, 26]。它在作品中作为 IBS 后端接入；Wang 等人的两层匿名 HIBE 与 IBS 通用构造则规定完整 IB-ME 协议的组合方式。

目标参数为

$$n = 512, \qquad q = 4294967291, \qquad s_0 = 4.$$

令 $k = \lceil \log_2 q \rceil = 32$，gadget 宽度 $w = nk = 16384$，公开矩阵宽度 $m = 32768$，签名向量维数 $M = 49152$。根公开矩阵与陷门满足

$$A = [\bar{A} \mid G - \bar{A}R].$$

身份密钥 X 满足 $AX + H_1 = G \pmod{q}$，签名短向量 z 满足

$$[A, H_1]z = H_2(P, \rho, s, \eta, M') \pmod{q}.$$

有限离散 Gaussian 由独立原生组件执行，Python 层负责域分隔哈希、参数快照、规范对象和精确模方程。本项目按约 115 bit 量子安全的工作假设使用该目标档；公钥约 64.0 MiB、主状态约 576 MiB、用户签名密钥约 2.06 GiB、签名约 384 KiB。该实现已完成独立签验、状态恢复以及 LATTE 组合，实验章单列其服务器资源和分阶段时间。

4.3 困难假设、参数档与来源对应

本作品保留各算法论文中的困难问题、参数档和安全模型。统一机器与输入用于比较实现成本，不改变这些配置各自的理论来源。表 4.1给出报告中实际使用的对应关系。

表 4.1 实际算法、困难假设与参数来源对应

算法后端	作品职责	困难问题与模型	采用配置与来源
LATTE-3	两层匿名 HIBE	NTRU 根陷门与 RLWE 封装；经典 ROM	$N = 1024$、$q = 68718428161$、深度 2，采用作者 LATTE-3 档[22]
Lai C2	两层 HIBE	PLWE（Ring-LWE 表示）与理想格陷门	$n = 2048$、$q = 274877976577$、$k = 39$、深度 2，采用 C2 冻结配置[23]
BNN / ML-DSA	认证转换 IBS	认证转换条件；ML-DSA 的 MLWE / MSIS 类模格问题	BNN 身份认证转换与 ML-DSA-44，普通签名格式遵循 FIPS 204[27, 28]
BNN / Falcon	认证转换 IBS	认证转换条件；Falcon 的 NTRU 格与 SIS 关系	BNN 身份认证转换与 Falcon-512 padded 格式[27, 36]
Luo	原生模格 IBS	MLWE 用于密钥恢复问题，MSIS 用于伪造问题；经典 ROM	$N = 256$、$(k, \ell) = (11, 10)$ 的作品冻结档[35]
SRGG	原生模格 IBS	MLWE / MSIS；QROM	论文 Table 4 的 PARAM-I：$n = 512$、模块秩 2、$Q = 15744063$[24]
Sageloli	原生环格 IBS	RSIS；ROM / QROM	论文 Table 6 的结构化 PARAM-I：$(d, k, \ell) = (2048, 65, 132)$[25]
Wang–PW	矩阵 SIS IBS	SIS；基本构造与自适应转换的随机预言机模型	目标档 $n = 512$、$q = 4294967291$、$s_0 = 4$[17, 26]

4.4 已实现算法与组合矩阵

两种 HIBE 与五种常规 IBS 形成表 4.2所示的十种组合。每一格均通过统一接口完成系统建立、双方发钥、消息加密、匹配解密和身份错配拒绝。同一算法的参数档或编码格式不重复计为新方案。

表 4.2 两种 HIBE 与五种常规 IBS 的端到端组合

HIBE	BNN + ML-DSA	BNN + Falcon	Luo	SRGG	Sageloli
LATTE	完成	完成	完成	完成	完成
Lai	完成	完成	完成	完成	完成

Wang–PW 在常规矩阵之外完成 LATTE + Wang–PW 端到端组合。当前已实现范围因此包括十种常规组合和这一套目标参数组合；Lai 与 Wang–PW 尚未组成完整运行记录。

4.5 代表方案的性能指标与选取依据

十种常规组合用于检验通用接口并比较成本分布，A、B、C 三组分别覆盖低延迟标准化签名路径、具有紧凑编码的原生模格 IBS，以及采用另一种 HIBE 与环格 IBS 的连续通信路径。LATTE + Falcon 与 A 具有相近的在线时间和更短的密文，但二者同属 BNN 认证转换路线，因此保留 ML-DSA 作为主演示，避免三组代表重复覆盖相同结构。表 4.3列出三组算法和容量。

表 4.3 三组代表的算法构成与应用消息容量

组	HIBE	IBS	应用消息上限
A	LATTE-3	BNN / ML-DSA-44	59,191 B
B	LATTE-3	SRGG PARAM-I 紧凑格式	65,364 B
C	Lai C2	Sageloli PARAM-I	3,924 B

表 4.4从第七章的 Ultra 9 275HX 十组同机基本比较中摘出三组代表。三组使用相同源码、运行时、线程配置、64 字节双方身份和 32 字节消息；每种组合建立一套独立系统及主密钥（下称 “独立根”），表中各项均为一次实测值。

表 4.4 三组代表的主要性能与资源指标

指标	A	B	C
首次加密	12.9 ms	17.6 s	96.4 s
首次解密	103 ms	125 ms	412 s
重复加密	13.5 ms	11.3 s	89.4 s
重复解密	59.4 ms	68.7 ms	409 s
恢复后解密	61.6 ms	68.2 ms	406 s
完整密文	139 KiB	146 KiB	4.34 MiB

“重复”表示同一进程、同一套已生成对象下的第二次操作；“恢复后解密”表示新进程加载规范对象后完成的解密。表中 C 组执行完整委派路径；会话路径的阶段数据在第五章和第七章单独列出。

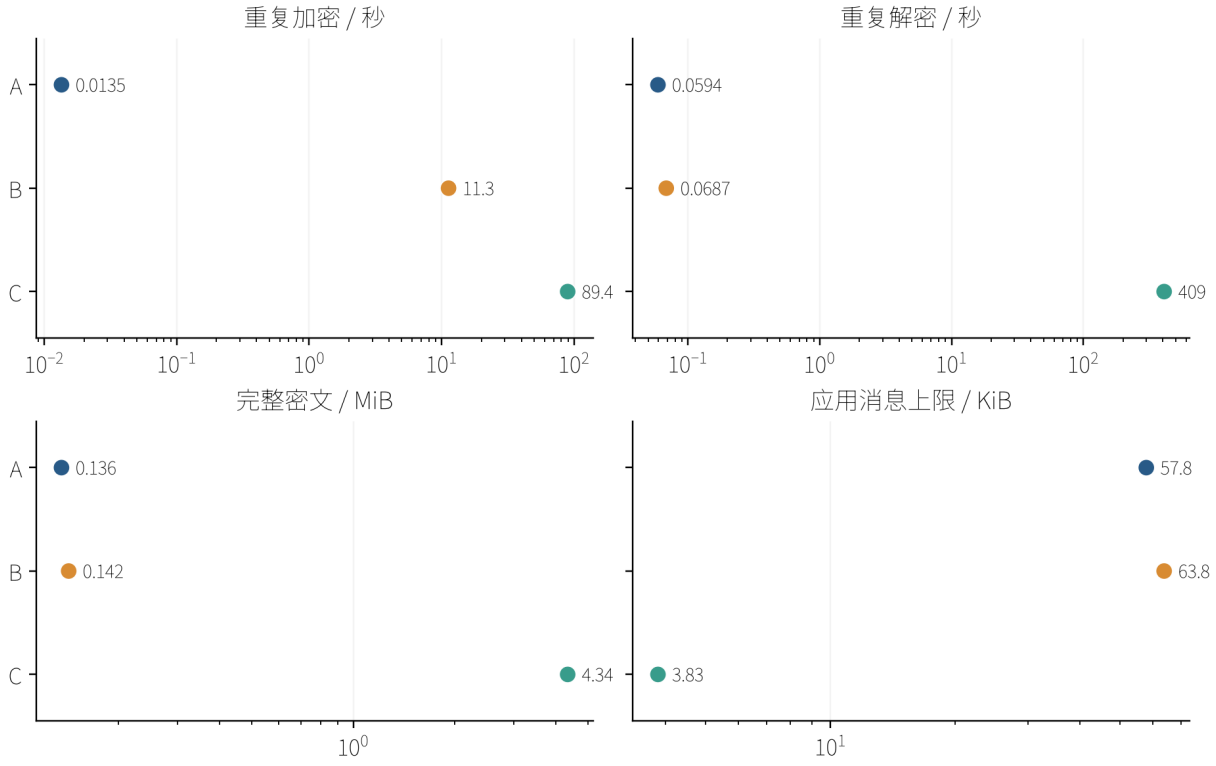


图 4.1 三组代表方案的同机时间、完整密文与应用容量；各面板使用原始量和对数刻度

同机结果使三组差异可以直接落到协议阶段。A 的首次和重复收发均处于亚秒级；B 的接收时间与 A 接近，主要增加发送者发钥和加密中的 SRGG 签名成本；C 的加密、解密和密文长度均明显增加，重点在于验证 Lai 与 Sageloli 能够进入同一完整协议，并为第五章的会话与结构化采样优化提供对象。十种组合的完整时间、对象尺寸和内存对照见第七章。

4.6 三组代表的应用定位

4.6.1 A 组：低延迟定向消息

A 组采用 LATTE-3 与 BNN / ML-DSA-44。其重复加密和解密均处于亚秒级，适合需要交互响应的机构消息、浏览器演示和频繁的小消息收发。ML-DSA 组件具有固定标准对象格式，认证转换也便于复用普通签名实现。A 组的主要通信开销来自一次性认证而不是 IBS 签名，因此后续优化重点是外层认证结构和整包长度。

4.6.2 B 组：原生模格身份认证与紧凑状态

B 组采用 LATTE-3 与 SRGG PARAM-I。其接收端时间与 A 接近，主要额外成本集中在发送者发钥和签名。范围感知编码显著减少 SRGG 主状态、用户密钥和签名长度。该组适合发送端能够承担十余秒签名、接收端需要快速验证的批量定向投递，也适合研究原生模格 IBS 和紧凑对象在完整协议中的作用。

4.6.3 C 组：固定身份对的连续通信

C 组采用 Lai C2 与 Sageloli PARAM-I。初次解密需要完成高成本叶钥委派，密文也明显大于 A、B；建立接收会话后，同一身份对的后续解密不再重复委派。因而，C 组更适合双方身份固定、允许离线准备并连续处理多条消息的使用方式，例如固定管理端与受控终端之间的周期消息。该组还展示了高成本 HIBE 与环格 IBS 如何通过结构化采样和会话复用进入完整系统。

4.7 本章小结

本章说明了两种 HIBE 与六种 IBS 的算法关系和实现对象，并区分十种常规组合与 LATTE + Wang–PW 目标参数组合。A、B、C 三组分别代表低延迟认证转换、原生模格身份签名和固定身份对的高成本环格路线。三组的差异同时体现在在线时间、对象尺寸和可复用的准备状态，不能由单一指标概括。

第五章　关键算法优化

第四章介绍的算法后端在结构、参数和对象规模上差异明显。直接照搬数学定义虽然可以描述算法，却不能解决目标参数运行时出现的内存、存储和重复计算问题。本章围绕三个实际瓶颈展开：Sageloli 相关扰动不能展开为稠密矩阵，SRGG 参考对象存在较大的文本表示开销，Lai 在固定身份对通信中会反复生成相同叶密钥。三项优化分别改变计算组织、对象编码和运行时生命周期，但不改变原方案的公开方程与公共协议接口。

5.1　Sageloli 结构化环采样优化

5.1.1　稠密矩阵展开的资源瓶颈

Sageloli 的身份发钥和签名都需要在格陪集中采样。相关扰动由陷门决定其形状，再与离散采样结合，生成满足方程和范数要求的短向量。环元素本来以多项式系数表示，乘法可以直接在负循环环上计算；若把每个环元素展开为旋转矩阵，陷门和协方差会随之变成规模很大的实数矩阵。

PARAM-I 取 $d = 2048$、$\ell = 132$、$k = 65$，签名相关扰动的系数维数为

$$D = d(\ell + k) = 403456.$$

仅显式保存一个 $D \times D$ 双精度实数矩阵就需要约 1.18 TiB，还没有计入矩阵分解和临时工作区。这一数量超过常规工作站的可用内存，因此优化的重点不是减少常数，而是避免形成该矩阵。

实现始终保留多项式结构。公钥、主陷门和身份陷门分别以 $\ell \times d$、$(\ell - k) \times k \times d$ 和 $\ell \times k \times d$ 的系数数组保存，环乘法在原环上执行。每次发钥和签名结束后仍以精确整数算术检查

$$A = [\bar{A} \mid g - \bar{A}R], \qquad AT_s = H_1(P, \rho, s) - g \pmod q,$$

因此改变的是内部计算方式，而不是密钥关系和验签条件。

5.1.2　逐频率低秩分解

设当前陷门为 T，其形状为 $\ell \times k$ 个环元素，并令

$$J = \begin{bmatrix} -T \\ I_k \end{bmatrix}, \qquad a = s^2 - r^2, \qquad c = 9r^2.$$

相关扰动所需协方差为

$$\Sigma = aI - cJJ^\star,$$

其中 $\star$ 表示环上的伴随。aI 是标量矩阵，修正项由只有 k 列的 J 生成。对任何与 $\mathrm{span}(J)$ 正交的向量，$J^\star x = 0$，所以修正项 $JJ^\star$ 不起作用，平方根算子只需在 J 张成的至多 k 维子空间内改变 $\sqrt{a}I$。这正是把大空间计算转移到低维子空间的依据。

首先对多项式执行负循环 Fourier 变换。在每个频率 ν 上，环乘法变为复数乘法，J 对应 $(\ell+k)\times k$ 复矩阵 J_ν。随后计算

$$G_\nu = J_\nu^* J_\nu = U_\nu \operatorname{diag}(\lambda_i) U_\nu^*.$$

在 PARAM-I 中，每个频率上需要分解的最大矩阵为 65×65，而不是 403456×403456。G_ν 的特征向量给出 $\operatorname{span}(J_\nu)$ 中的正交方向，特征值则给出各方向上需要从 a 减去的修正量；在该子空间的正交补上，平方根作用仍是简单的 $\sqrt{a}$ 倍缩放。

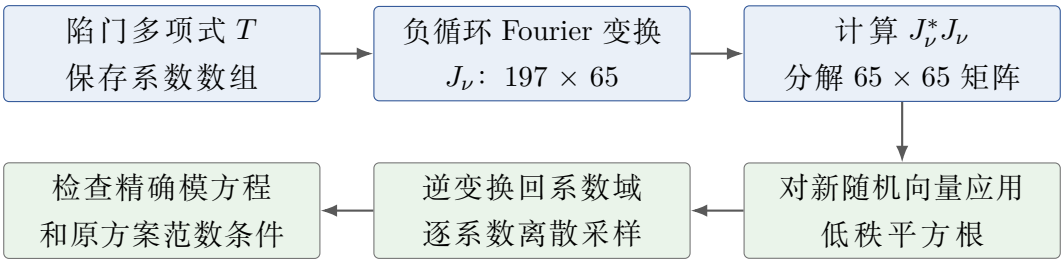


图 5.1 Sageloli 结构化相关扰动的计算流程（矩阵形状对应 PARAM-I）

根据小矩阵的特征分解构造平方根算子

$$L_\nu = \sqrt{a} I - J_\nu U_\nu \operatorname{diag}\left(\frac{c}{\sqrt{a}+\sqrt{a-c\lambda_i}}\right) U_\nu^* J_\nu^*.$$

该算子满足 $L_\nu L_\nu^* = aI - cJ_\nu J_\nu^*$。应用算子时，程序先把新生成的标准随机向量投影到 J_ν 的列空间，再用 65×65 特征分解修正这些分量；其余分量直接乘以 $\sqrt{a}$。整个过程只执行小矩阵与向量乘法，不形成完整 L_ν。逆变换回系数域后，继续执行原方案要求的离散 Gaussian 和三进制陪集采样。该方法沿用格陷门采样中的扰动思想[29]，并针对当前方案的环结构组织计算。

主要存储量由陷门系数和逐频率小矩阵计划组成，规模为 $O(d\ell k + dk^2)$，不再随 D^2 增长。固定陷门的系数数组、Fourier 表示和各频率特征分解可以复用；每次发钥或签名仍重新生成 Gaussian 源向量、相关扰动和陪集样本。表 5.1比较两种计算组织。

表 5.1 Sageloli 稠密表示与结构化实现对照

计算组织	核心分解规模	主要存储	PARAM-I 结果
系数空间稠密展开	一个 403456×403456 实矩阵	单个双精度矩阵约 1.18 TiB，尚未计分解工作区	资源估算，不执行展开
逐频率低秩实现	2048 个 197×65 的 J_ν，各分解 65×65 的 Gram 矩阵	$O(d\ell k + dk^2)$ 系数与数值计划	完成发钥、签名和公开验证，实测峰值 RSS 约 1.36 GiB

Sageloli 原论文给出 SIS / RSIS 两类 IBS 构造、ROM / QROM 分析以及 Table 6 参数[25]。本作品围绕其中 RSIS 参数补充系数原生对象、逐频率低秩 SampleF、目标尺寸编解

码和精确环方程检查，并把后端接入通用 IB-ME 流程。这里的工程贡献是把论文参数组织成可运行、可保存的结构化实现，公开验签关系和困难假设仍采用原方案。

5.1.3 数值采样与精确代数校验

Fourier 变换和特征分解使用数值运算，方案中的模方程则必须精确成立。实现将两类职责分开：数值计划在建立时检查正定充分条件和变换误差；回到系数域后，以整数算术检查环方程。大模数 $q = 3^k$ 的公开乘积使用 FLINT 整数多项式，私有小整数乘法采用双素数 NTT 与 CRT 恢复，并检查整数卷积幅度处于可唯一恢复范围。

签名生成后继续执行原方案的范数门和验签方程。浮点计算负责生成相关扰动，精确算术负责确认最终代数关系。第七章给出 PARAM-I、PARAM-II 的实测时间、峰值内存及验签结果。

5.2 SRGG 对象的范围感知编码

5.2.1 参考表示的空间开销

SRGG 的公钥、陷门和签名包含大量多项式系数。JSON 会把每个整数写成十进制字符串，并重复保存负号、逗号、括号和字段名。对于大对象，这些语法字符和文本数字会形成显著的额外开销。

紧凑编码依据各字段的数学取值范围选择位数。例如，取值为 $-1, 0, 1$ 的陷门系数只有三种可能，每项用 2 bit 即可表示；公开模 $Q = 15744063$ 的系数位于 $[0, Q)$，每项使用 24 bit。短密钥和签名响应先以 Zigzag 映射处理符号，再选择能够容纳该字段的最小统一位宽。

表 5.2 SRGG 不同字段的紧凑编码方法

数据	存储方法	解码时检查
公开模 Q 系数	每系数 24 bit	$0 \le x < Q$
三值陷门系数	每系数 2 bit	拒绝第四个未使用码字
短密钥、响应整数	Zigzag 与最小统一位宽	数值范围、位宽最小性
二元挑战	每系数 1 bit	固定多项式次数与长度
类型、参数与上下文	固定头部和长度前缀	对象类别、公钥和参数一致

对包含 N 个系数、统一位宽为 b 的字段，主体长度为 $\lceil Nb/8 \rceil$ 字节，再加必要头部。解码后得到与 JSON 格式相同的数学对象，签名与验证仍调用原算法。

5.2.2 四类对象的规范编解码

范围感知编码同时用于公钥、主状态、用户密钥和签名。四类对象使用独立类型标识，系数顺序固定，末字节中未使用的位必须为零。解析器先根据参数和形状计算预期长度，再读取数据，并检查对象类型、身份、公钥上下文和系数范围。解码后重新编码必须得到相同字

节串，从而排除同一数学对象的多种字节表示。

对象变短可以减少密钥文件和传输字节，但收益取决于对象在完整协议中的占比。例如，IBS 签名变短会直接减少对称加密载荷，而 HIBE 胶囊和一次性认证仍构成固定开销。第七章同时报告四类对象的 JSON 与紧凑格式长度、编解码时间以及完整密文组成，以区分对象级收益和协议级收益。

SRGG 原论文给出 PARAM-I、II、III 三档规范、QROM 安全分析和概念验证实现[24]。本作品在此基础上为公钥、主状态、用户密钥和签名建立统一的无损规范编码，补充严格解析、状态持久化、通用 IB-ME 接入及完整载荷核算。表 5.3使用同一 PARAM-I 对象的两份已有功能记录，比较参考表示与紧凑表示；这是一项基于既有记录的协议载荷核算。

表 5.3 SRGG PARAM-I 编码对完整 LATTE + SRGG 载荷的影响

对象	参考表示	紧凑表示	减少
SRGG 签名	35,289 B	13,400 B	21,889 B（62.0%）
完整 B 密文	171,114 B	149,231 B	21,883 B（12.8%）

两份记录均使用 LATTE、SRGG PARAM-I、10 字节消息及同一协议格式；数值来自已有配对功能记录，不计作新的性能实验。

5.3 Lai 接收会话优化

5.3.1 固定身份对下的重复委派

在 Lai 组合中，接收者持有第一层密钥 dk_r。解密来自发送者 s 的消息前，需要运行

$$\mathsf{dk}_{r,s} \leftarrow \mathsf{Derive}(\mathsf{dk}_r, s).$$

同一接收者连续读取同一发送者的消息时，委派输入保持不变，逐消息生成叶钥会重复支付相同的计算成本。与此同时，每条消息的 HIBE 胶囊、对称密文和 IBS 签名都不同，仍须逐条认证与解密。

作品引入显式接收会话：会话建立时生成并验证一次叶钥，随后在固定双方身份下复用。图 5.2对比两种计算安排。

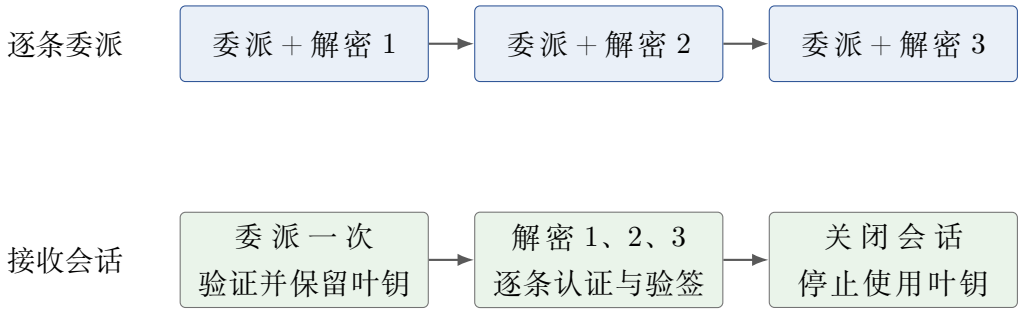


图 5.2 Lai 接收会话将与密文无关的委派移至准备阶段

5.3.2 身份绑定与会话生命周期

会话建立时先检查公开上下文、接收者身份和一级密钥绑定；生成叶钥后再检查完整 (r, s) 路径及陷门方程。会话只服务于当前公钥和身份对，关闭后停止解密。每条消息继续执行规范解析、外层字节认证、HIBE 解封装、AES-GCM 验证、IBS 验签和一次性公钥承诺核对。

叶钥作为秘密状态仅保留在进程内，会话对象不支持序列化；长期保存的接收密钥继续采用原有持久格式。该边界避免为了性能而增加新的长期密钥文件，同时明确会话优化适用于双方身份固定的连续通信。

5.3.3 连续消息的摊销成本

设一次会话准备耗时为 T_{prep}，会话内单条解密耗时为 T_{sess}。处理 N 条消息的总成本和平均成本分别为

$$T_{\mathrm{total}}(N) = T_{\mathrm{prep}} + NT_{\mathrm{sess}}, \qquad \bar{T}(N) = \frac{T_{\mathrm{prep}}}{N} + T_{\mathrm{sess}}.$$

与每条消息均重新委派的原路径 NT_{original} 相比，会话累计成本更低的条件为

$$N > \frac{T_{\mathrm{prep}}}{T_{\mathrm{original}} - T_{\mathrm{sess}}}.$$

现有阶段中位时间为 $T_{\mathrm{prep}} = 509.548$ 秒、$T_{\mathrm{original}} = 526.272$ 秒、$T_{\mathrm{sess}} = 28.568$ 秒，代入得到 $N > 1.024$，因此最小整数临界点为 2 条消息。该结论是基于现有中位时间的成本模型；第七章另列实际测量次数和 36,414,052 字节的叶钥编码长度。

5.4 本章小结

三项优化分别针对不可承受的中间矩阵、冗长的对象表示和固定身份对下的重复委派。Sageloli 优化通过逐频率低秩分解避免 $O(D^2)$ 稠密表示；SRGG 优化按字段范围建立规范二进制编码；Lai 优化以受身份和公钥约束的会话复用叶密钥。它们都保留原算法方程与通用协议接口，具体效果在第七章以实测数据验证。

第六章　软件系统与操作流程

密码原语解决具体数学问题，软件系统还要负责组合选择、参数检查、对象保存和用户操作。本章介绍系统怎样把后端、公共协议和应用入口分开，以及新增 HIBE 或 IBS 时需要完成哪些工作。

6.1　分层软件架构

系统自下而上分为基础设施层、密码原语层、组合层、协议层和应用层。基础设施层提供规范编码、随机源、域分隔哈希、错误类型和原子文件保存。密码原语层分别实现 HIBE 和 IBS。组合层根据配置选择后端，并在生成密钥前完成能力与容量检查。协议层实现第三章的五个 IB-ME 算法。最上层提供命令行、实验工具和浏览器演示。

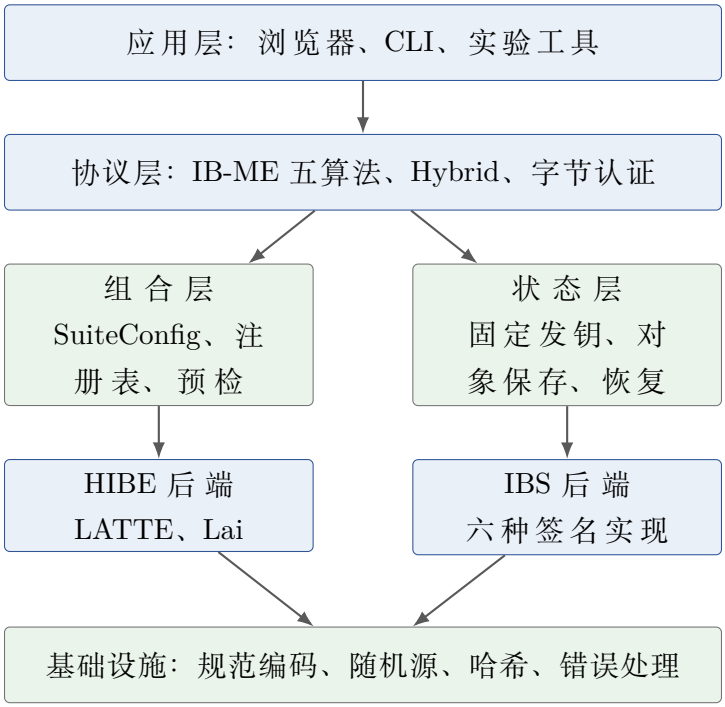


图 6.1　软件系统的分层结构

这一结构有两个直接作用。第一，公共协议不根据算法名称编写分支。它只调用接口，并从组合配置读取容量和格式。第二，应用入口不直接操作陷门、矩阵或多项式；命令行和浏览器收到的都是统一角色对象和密文字节。因此，同一套页面、状态管理和错误处理可以服务不同组合。

6.2 稳定的原语接口

HIBE 后端提供五个密码方法,IBS 后端提供四个密码方法和一个签名加载方法。表 6.1列出接口及其返回对象。

表 6.1 HIBE 与 IBS 后端接口

后端	方法	公共含义
HIBE	setup	建立公开参数与主秘密
HIBE	extract	为第一层接收身份生成密钥
HIBE	derive	为第二层发送身份委派叶密钥
HIBE	encrypt	沿两层身份路径加密固定长度种子
HIBE	decrypt	用叶密钥恢复种子或返回拒绝
IBS	setup	建立签名公开参数与主秘密
IBS	key_extract	为发送身份生成签名密钥
IBS	sign	签署规范协议消息
IBS	verify	以公开参数、身份和消息验签
IBS	load_signature	从规范字节恢复签名对象

每种公开参数、主秘密、用户密钥、密文和签名都实现规范的 to_bytes 与 from_bytes。对象头包含类别、版本和参数摘要。后端加载对象时检查字段数量、长度、数值范围和公开上下文，公共层无需了解其中的系数排列。

接口也明确随机源的传递方式。建立、发钥、加密和随机化签名从上层取得密码学随机源；验签与规范解码不读取随机数。测试可以注入可重复随机流，正常运行使用操作系统随机源。同一方法在不同入口下因此保持相同的随机性边界。

6.3 组合配置与接入流程

6.3.1 完整组合配置 SuiteConfig

一套运行配置由 SuiteConfig 描述。它记录 HIBE 与 IBS 选择器、算法版本、两边参数档、密钥和签名格式、外部身份长度、混合载荷预算以及协议版本。配置使用排序字段和规范 JSON 编码，其摘要进入公开对象和所有角色密钥。

注册表为每个后端保存四类信息：构造函数、参数预设、对象解码器和能力声明。能力声明包括两层身份支持、内部身份宽度、直接明文容量、IBS 输入上限、签名尺寸上界、所需运行依赖和允许的编码格式。发现后端和查看配置只读取这些静态信息，不生成密钥。

运行前的 validate_suite 依次完成：

1. 解析 HIBE 与 IBS 选择器，固定精确版本和参数档；

2. 检查 HIBE 支持两层身份，外部身份编码后不超过内部上限；

3. 计算 IBS 签名输入开销和最大应用消息长度；

4. 检查签名上界与应用消息能够放入混合载荷预算；

5. 检查对象格式和运行依赖，生成完整配置摘要。

只有配置通过以后，系统才进入 Setup。容量不足会在昂贵建立之前给出所需字节数和现有预算，用户可以选择更大载荷预算或另一后端。

6.3.2 新增 HIBE 或 IBS 的接入步骤

接入新 HIBE 时，实现者完成五个 HIBE 方法和五类对象，声明两层身份宽度、种子容量和参数预设，再向注册表提供一个后端描述。接入新 IBS 时，实现四个密码方法、签名加载和四类对象，声明身份、签名输入及签名输出上限。公共协议、Hybrid、BYTEAUTH、状态文件和应用入口均不修改。

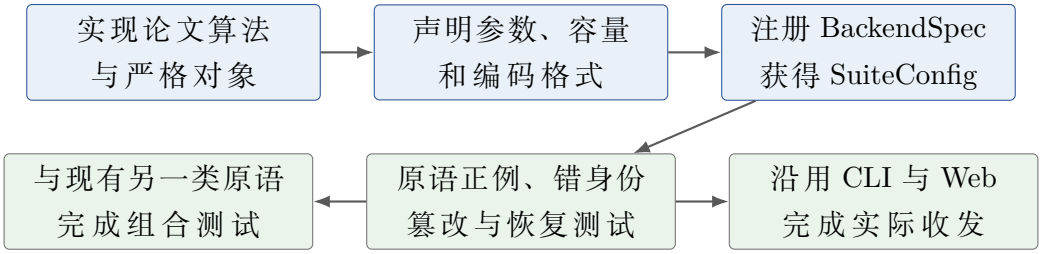


图 6.2 新增密码后端的接入过程

以 Wang–PW 为例，新增代码位于独立 IBS 目录，向外暴露与 Sageloli、SRGG 相同的方法。注册描述声明目标档 s0-4、393,320 字节签名和 4,096 字节签名输入上限。组合层据此算出 3,924 字节应用容量，并把混合载荷预算提高到能够容纳该签名的范围。随后直接复用 LATTE、五个 IB-ME 算法、命令行角色导入和错误输入流程；公共协议仍使用统一接口和处理顺序，不增加后端专属分支。

这一接入方式也是“通用设计”的软件依据。更换算法需要完成新原语本身及其元数据，而不是复制一套 IB-ME 实现。十种常规配对和 LATTE + Wang–PW 都经过同一入口形成。

6.4 固定发钥与持久状态

现实系统不会为每条消息重新建立主密钥。发钥机构需要长期保存主状态，并保证同一身份能够取回先前发布的密钥。作品为 HIBE 接收者和 IBS 发送者分别维护发钥记录。

一次发钥分为准备、生成和提交三个阶段。准备阶段取得本地锁并写入待处理身份；生成阶段调用底层原语；提交阶段先原子保存更新后的主状态，再发布用户密钥。若进程在生成中断，恢复程序会看到待处理记录并停止继续发钥，避免同一身份在状态不明时获得第二份对象。

加载状态时，系统按图 6.3检查组合配置、公开上下文、参数快照、对象类型和身份绑定。后端随后检查自己的代数关系，例如 LATTE 叶钥方程、SRGG 对象形状或 Wang–PW 的 $AX + H_1 = G$ 关系。通过后才恢复运行实例。

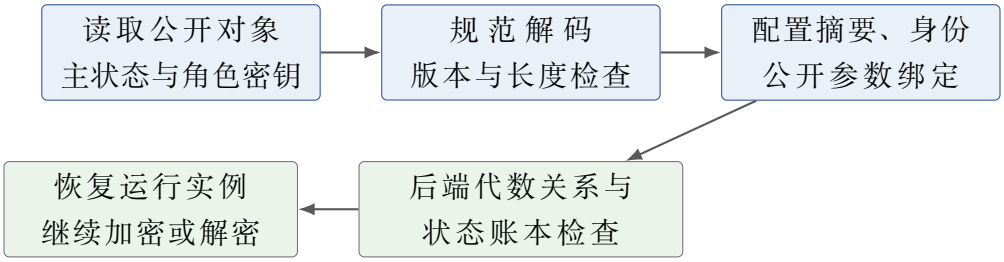


图 6.3 持久对象恢复流程

公开对象、发送密钥、接收密钥和密文可以分别导出。角色导入只要求该角色实际需要的对象：发送端不读取 HIBE 主秘密，接收端不读取 IBS 主秘密。这样可以在不同进程甚至不同主机上执行发送和接收，同时保持相同的组合配置与编码。

6.5 命令行与浏览器操作

命令行适合批量实验、持久状态和跨进程恢复。典型流程为：选择组合并建立系统；为 Alice、Bob 导出角色密钥；Alice 指定 Bob 并加密；Bob 指定 Alice 并解密；关闭进程后重新导入对象，再次解密已有密文。实验驱动在此基础上记录每阶段时间、对象字节数和进程树 RSS。

浏览器界面使用同一运行时，不另写一套密码代码。使用者首先选择组合，然后输入接收者、发送者、期望发送者和消息。页面显示建立、发钥、签名、封装和解密阶段，并展示密文长度、耗时和最终结果。对于耗时较长的 B 组，任务在服务端继续执行，页面刷新后可以重新连接正在运行的任务。



图 6.4 A 组实际收发界面：指定双方身份、加密消息与匹配解密

图 6.4记录 A 组的一次实际操作：Alice 向 Bob 发送 36 字节 UTF-8 消息，Bob 选择 Alice 为期望发送者后恢复原文。A、B 还分别在同一条密文上执行三种身份错配、翻转 1 bit 和截短 1 字节，均得到拒绝。两组该次密文长度分别为 142,095 字节和 149,255 字节。



(a) 身份匹配并恢复原文



(b) 接收身份不匹配并拒绝

图 6.5 网页演示关键结果局部：发送者、接收者、期望发送者与解密结果

6.6 运行优化与部署方式

除了第五章的三项主要优化，系统还减少两类重复工作。BNN 签名包含机构证书和用户消息签名。对同一身份的连续消息，系统缓存“证书已经验证”的公开结果，但每条消息仍重新验证用户签名。缓存绑定完整证书、用户公钥和身份，并使用有界 LRU 管理。

SRGG 签名的一部分数值准备只取决于固定签名密钥和数值选项。实现可以复用该计划，每条签名仍重新生成随机量。实验表明这两类局部缓存对完整流程的影响小于结构化采样、紧

凑编码和 Lai 会话，因此它们作为运行细节保留。

系统支持单机演示和分角色部署。单机模式把发钥机构、发送端、接收端和浏览器服务放在一台主机，便于比赛现场操作。分角色模式由发钥机构离线生成角色对象，发送端和接收端分别导入自己的密钥；密文可通过任意字节传输通道送达。

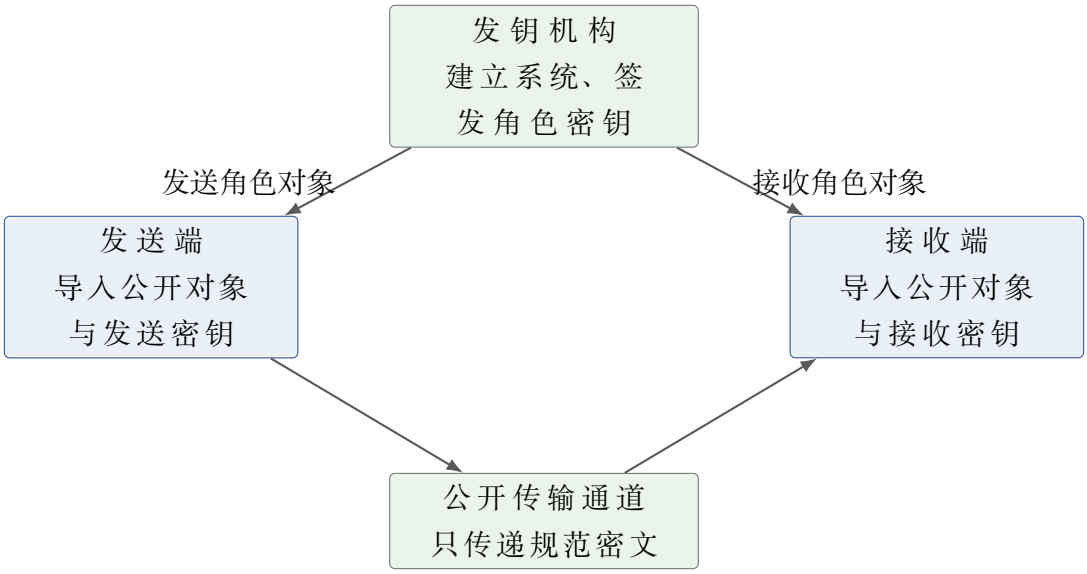


图 6.6 发钥机构、发送端与接收端的部署关系

原生模块在创建适配器时会检查接口和二进制。已有部署对照发现，把安装包和运行文件放在 Linux 本地文件系统可以减少跨挂载目录的重复读取。该调整不改变算法、参数和密文格式，第七章列出同一运行时下的对照结果。

除主要实验机器外，项目还在 Xeon Silver 4216 主机的 NixOS、Sage 10.9 和 Python 3.14.7 环境完成独立安装。A、B 各执行 25 条操作，C 完成匹配解密、内部签名拒绝和恢复。C 最初因 900 秒阶段上限导致超时；调整为 3600 秒后完成三个长阶段，耗时约 21 至 27 分钟。这次复现用于检查安装、对象恢复和完整调用路径；性能分析仍采用第七章逐项列明的原始实验环境。

6.7 本章小结

软件系统通过稳定接口、组合配置和注册表把底层算法与公共协议分开。新增后端只实现自身算法、对象与能力声明，随后即可复用五个 IB-ME 算法、状态管理、CLI 和浏览器入口。固定发钥和规范恢复使系统能够跨进程长期使用，单机与分角色部署则覆盖演示和实际通信两种方式。

第七章　测试与结果分析

7.1　实验设置

实验分别考察完整系统的运行成本、三项主要优化、部署改动和功能正确性。数学基础见第二章，具体算法与参数见第四章；身份和消息采用规范字节编码，正常密码操作使用操作系统随机源。十种常规组合的主比较全部采用表 7.1中的统一环境，专项实验环境在相应小节另列。

表 7.1　十种常规组合的同机实验环境与方法

项目	统一条件
硬件与系统	Intel Core Ultra 9 275HX；WSL2 Linux；固定逻辑 CPU 0–3
运行时与线程	Sage 10.9、Python 3.13.14；OMP 4，OpenBLAS、MKL 与 NUMEXPR 均为 1 线程
输入	发送与接收身份各 64 B，应用消息 32 B，系统随机源
重复方式	每种组合建立 1 个独立根；首次、同进程重复和新进程恢复后操作各 1 次
记录项目	各阶段墙钟时间、规范对象与完整密文字节数、进程树采样 RSS、诚实结果与外层拒绝

完整流程计时包含组合编码、对象检查和原语调用。首次解密包含叶钥生成；恢复实验另起进程加载已有对象。十种组合的实验固定逻辑 CPU 0–3，OpenBLAS、MKL 与 NUMEXPR 均为 1 线程，首次操作在运行库准备后开始计时。表中重复测量描述固定密钥下的运行波动，独立建立系统的数量单独列出。下文时间通常保留 3 位有效数字；B、KiB、MiB 分别表示字节、2^{10} 字节和 2^{20} 字节。

7.1.1　组合覆盖与代表方案

第四章表 4.2已经列出两种 HIBE 与五种常规 IBS 形成的十种组合。测试阶段对每一格都调用真实后端，完成系统建立、双方发钥、首次与重复消息收发、新进程恢复和外层篡改拒绝。组合数量按底层算法计数，同一算法的不同参数档及参考 / 紧凑格式不重复计为新方案。

十种组合形成常规后端的主要横向比较；A、B、C 在此基础上承担更深入的重复、阶段分解和专项优化分析。Wang–PW 是在常规十种组合之后接入的第六种 IBS，目前完成 LATTE + Wang–PW 目标档，其输入长度、运行机器和资源规模不同，因而以独立的分阶段结果呈现。

7.1.2 评价指标

实验按“离线准备、在线收发、持久恢复”三阶段记录。离线准备包括系统建立与双方发钥；在线收发包括首次加密、首次解密和固定密钥下的重复操作；持久恢复另起进程加载公开对象与角色密钥。时间采用墙钟时间，尺寸采用实际规范编码字节数，内存采用定期采样的进程树 RSS 峰值。

这些指标对应不同部署选择。发钥机构更关心 Setup、KeyExtract 和主状态，发送端更关心签名与完整加密，接收端更关心叶钥委派和解密，网络侧则关心完整密文而不是单个签名。后文先分析完整系统，再把总成本分解到具体优化。

7.2 十种常规组合的同机基本比较

7.2.1 实验方法与完成情况

十种组合使用同一份固定源码归档、相同原生引擎和依赖版本，按串行方式运行。三组来自同口径预运行，另外七组在相同环境下补齐；两批的源码摘要、软件环境、线程设置和输入完全一致。每种组合建立一套独立系统及主密钥，依次执行首次加解密、同进程重复加解密、新进程对象恢复和恢复后解密，并分别检查密文截断和单比特翻转。

十种组合全部完成，共通过 20 次诚实加密、30 次诚实解密、10 次新进程恢复和 20 次外层拒绝，没有失败记录。每项时间为一个独立根上的一次实测。所用配置均为第四章列出的目标或论文参数而非教学参数；哈希与随机预言机接口使用 SHAKE 实例化，ROM 或 QROM 适用范围沿用相应原语文献，功能与性能测试不替代安全性论证。

7.2.2 首次完整流程时间

表 7.2按同一口径列出从系统建立到首次解密的五个阶段。系统建立和双方发钥主要反映可信发钥机构的准备成本，首次加密与首次解密则包含完整协议编码、签名、HIBE 封装、叶钥生成和认证检查。

表 7.2 十种组合的首次完整流程实测时间，单位为秒

组合	建立	发送者发钥	接收者发钥	首次加密	首次解密
LATTE + ML-DSA	0.729	0.0139	0.224	0.0129	0.103
LATTE + Falcon	0.689	0.0229	0.34	0.0153	0.0951
LATTE + Luo	0.819	1.64	0.37	0.974	0.2
LATTE + SRGG	1.08	33.3	0.385	17.6	0.125
LATTE + Sageloli	3.11	50.9	9.98	12.9	0.616
Lai + ML-DSA	1.95	0.549	30.6	86.5	422
Lai + Falcon	2.14	0.621	32	90.9	440
Lai + Luo	2.14	0.8	31.2	85	427
Lai + SRGG	2.26	28.1	30.7	91.2	428
Lai + Sageloli	4.6	49.9	39.5	96.4	412

LATTE 与 ML-DSA、Falcon 组合的首次加密分别为 12.9 和 15.3 毫秒；更换为 Luo、SRGG 和 Sageloli 后，首次加密分别为 0.974、17.6 和 12.9 秒，差异主要来自签名所需的数值准备、随机采样和对象处理。发送者发钥是表中的独立阶段，其差异由“发送者发钥”列反映。LATTE 五组首次解密为 95.1 至 616 毫秒，说明在接收 HIBE 不变时，IBS 验证虽有影响，但不会改变整体量级。

Lai 五组首次加密集中在 85.0 至 96.4 秒，首次解密集中在 412 至 440 秒。更换 IBS 后结果仍处于同一量级，表明该配置下 Lai 封装与真实随机委派是在线收发的主要成本。Sageloli 使发送者发钥进一步增至约 50 秒；IBS 自身代价应结合表中独立的发送者发钥列判断。

7.2.3 重复操作与新进程恢复

图 7.1同时展示首次、同进程重复和恢复后操作。横轴采用对数尺度，以容纳毫秒级 LATTE 认证转换组合和分钟级 Lai 组合；每个点对应一个独立根上的一次实测。

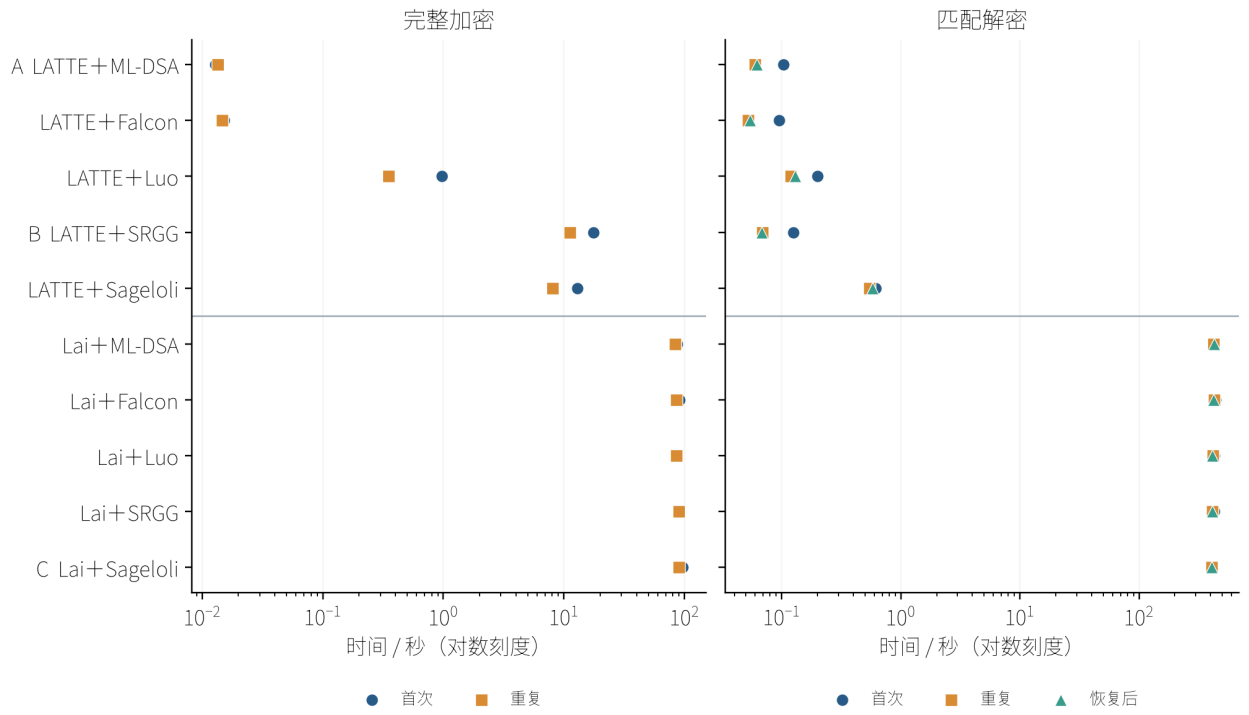


图 7.1 十种组合的首次、重复与恢复后完整操作时间；A、B、C 为三组代表

LATTE + Luo、SRGG 和 Sageloli 的重复加密分别降至 0.351、11.3 和 8.04 秒，与相同发送密钥下可以复用部分数值准备的实现结构一致。LATTE 五组恢复后解密为 54.5 至 577 毫秒，和同进程重复解密接近，说明规范对象加载没有改变接收路径的量级。Lai 五组恢复后解密仍为 406 至 428 秒，因为恢复不会缓存新的随机委派结果；这与第五章另行讨论的“准备接收会话”是两种不同状态策略。

7.2.4 规范对象尺寸与运行内存

表 7.3列出同一系统根产生的公共对象、主状态、双方角色对象和完整密文。尺寸均取实际规范二进制编码长度，不使用数学对象的内存估算。

表 7.3 十种组合的规范对象与完整密文长度

组合	公共对象	主状态	发送钥	接收状态	完整密文
LATTE + ML-DSA	14.1 KiB	142 KiB	7.86 KiB	125 KiB	139 KiB
LATTE + Falcon	13.7 KiB	137 KiB	4.1 KiB	125 KiB	135 KiB
LATTE + Luo	255 KiB	479 KiB	306 KiB	125 KiB	148 KiB
LATTE + SRGG	46.2 KiB	231 KiB	82.4 KiB	125 KiB	146 KiB
LATTE + Sageloli	3.36 MiB	149 MiB	137 MiB	125 KiB	3.21 MiB
Lai + ML-DSA	1.54 MiB	3.86 MiB	7.86 KiB	19.5 MiB	1.27 MiB
Lai + Falcon	1.54 MiB	3.85 MiB	4.1 KiB	19.5 MiB	1.27 MiB
Lai + Luo	1.78 MiB	4.19 MiB	306 KiB	19.5 MiB	1.28 MiB
Lai + SRGG	1.58 MiB	3.95 MiB	82.3 KiB	19.5 MiB	1.28 MiB
Lai + Sageloli	4.9 MiB	153 MiB	137 MiB	19.5 MiB	4.34 MiB

接收状态几乎完全由 HIBE 决定：LATTE 五组约为 125 KiB，Lai 五组约为 19.5 MiB。发送钥则主要由 IBS 决定，同一种 IBS 分别搭配两种 HIBE 时长度基本一致。完整密文也呈现可分解的结构：固定 IBS 后，由 LATTE 换为 Lai 会增加约 1.137 MiB；采用 Sageloli 时，大型 IBS 签名又使完整密文达到 3.21 和 4.34 MiB。该规律与后文按字段核算密文组成的结果一致。

图 7.2把完整密文与运行期间的最大采样 RSS 并列。除 Sageloli 外，十种组合各阶段最大采样 RSS 约为 254 至 885 MiB；LATTE + Sageloli 约 1.64 GiB，Lai + Sageloli 约 1.97 GiB。RSS 每 10 毫秒采样一次，包含运行库与子进程，共享页可能重复计数，因而用于同批次资源比较；它记录的是进程树采样值，并非物理页的精确分解。

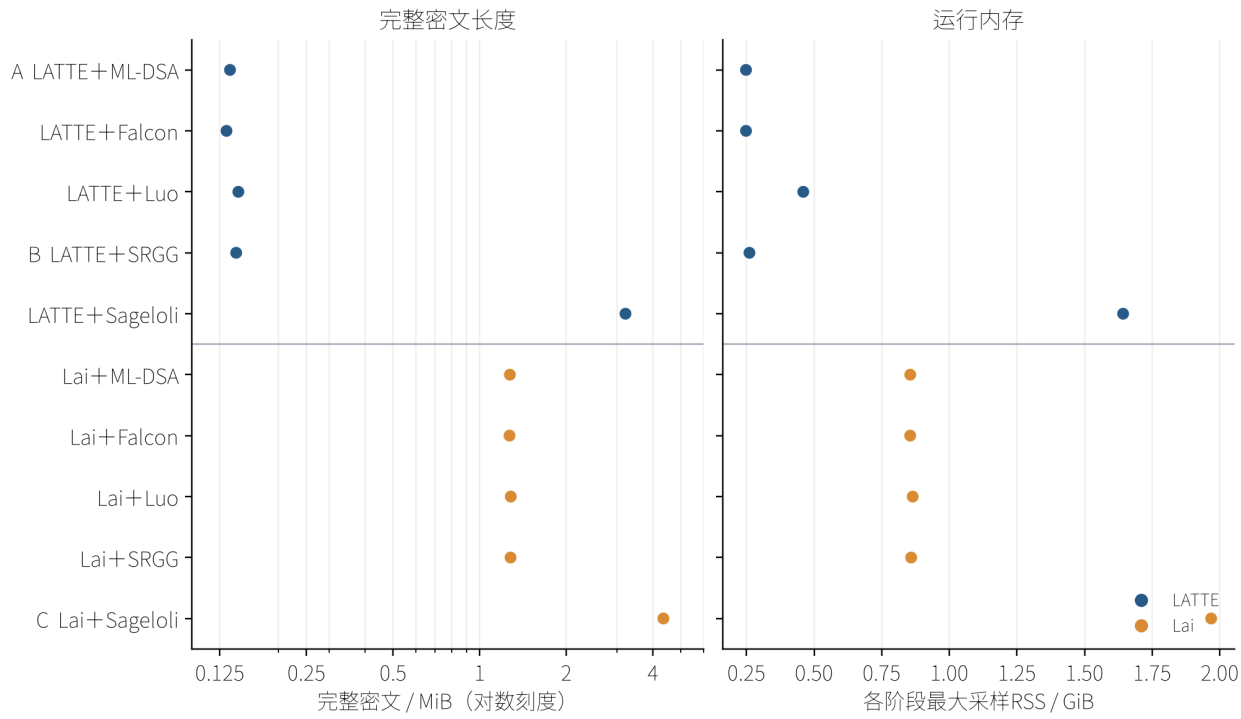


图 7.2 十种组合的完整密文长度与各阶段最大进程树采样 RSS

7.2.5 组合差异与代表方案

横向结果表明，HIBE 和 IBS 影响完整系统的不同位置。HIBE 主要决定接收状态、HIBE 胶囊和解密委派；IBS 主要决定发送者发钥、签名生成、发送钥和签名字段。采用 LATTE 时，五种 IBS 的发送差异可以直接观察；采用 Lai 时，HIBE 成本占主导，但不同 IBS 仍通过密钥和密文长度体现。因而系统选型不宜把全部指标压缩成单一总分，而应分别考虑发钥机构、发送端、接收端和网络侧资源。

三组代表由此对应三种明确用途。A 提供低延迟标准化签名路径；B 保留快速接收，同时展示原生模格 IBS 和紧凑编码的发送端代价；C 覆盖另一 HIBE 与环格 IBS，并为结构化采样和固定身份对会话优化提供完整对象。第四章的代表方案表采用本节同机数据；下面的多根与阶段实验进一步分析运行波动和瓶颈组成。

7.3 代表方案的重复与专项分析

专项实验用于分析运行波动、单项优化和内部阶段，所用机器、输入与重复方式列于表 7.4。这些数据按各自实验条件单独呈现；表 7.1 中的十种组合单根实测继续作为统一横向比较。

表 7.4 代表方案与优化专项实验的环境及用途

实验	环境	输入、重复与用途
A/B 多根重复	i9-14900HX，Sage 10.7 / Python 3.12.13	每组 3 个独立根，每根 3 对重复操作；观察根间与根内波动
Sageloli 采样	Ultra 7 255HX，Python 3.13.15	PARAM-I、II 各 1 套；测量结构化发钥、签名和验证
SRGG 编码	Ultra 7 255HX，Python 3.13.15	每档 3 套；同一对象两种格式各交替测量 5 次
C 阶段分解	Ultra 9 275HX，Sage 10.9 / Python 3.13.14	C 组合 1 次，分解委派与后续阶段
Lai 接收会话	i9-14900HX，Sage 10.7 / Python 3.12.13	1 套系统；原路径 2 次、准备 1 次、会话内解密 3 次
部署位置	Ultra 9 275HX，Python 3.13.14	相同 3 套状态；每种位置各 30 次加密和解密

7.3.1 A/B 多根重复实验

A、B 使用相同 LATTE 参数与相同输入，主要差别在 IBS。该批实验在另一台机器上分别建立 3 个独立系统根，每根执行 3 次重复操作，用于观察固定组合的运行波动。表 7.5给出完整生命周期的中位时间，图 7.3展示重复加解密的 9 次观测。该批结果用于补充多根运行波动，前述单根同机结果用于组合间横向比较。

表 7.5 A/B 完整流程中位时间，单位为秒

操作	A	B	测量次数（A/B）
系统建立	1.12	1.6	3/3
发送者发钥	0.0954	32.1	3/3
接收者发钥	0.427	0.457	3/3
首次加密	0.0963	9.14	3/3
首次解密	0.284	0.239	3/3
重复加密	0.0785	9.17	9/9
重复解密	0.152	0.161	9/9
对象恢复	0.845	0.89	3/3
恢复后解密	0.154	0.134	9/9

两组接收者发钥和重复解密接近，额外成本主要出现在 B 的发送者发钥与签名生成。A、B 重复加密中位数分别为 78.5 毫秒和 9.17 秒，重复解密分别为 152 毫秒和 161 毫秒。A 适

合作为交互演示基线；B 展示原生模块格 IBS 在同一完整协议中的运行特点。

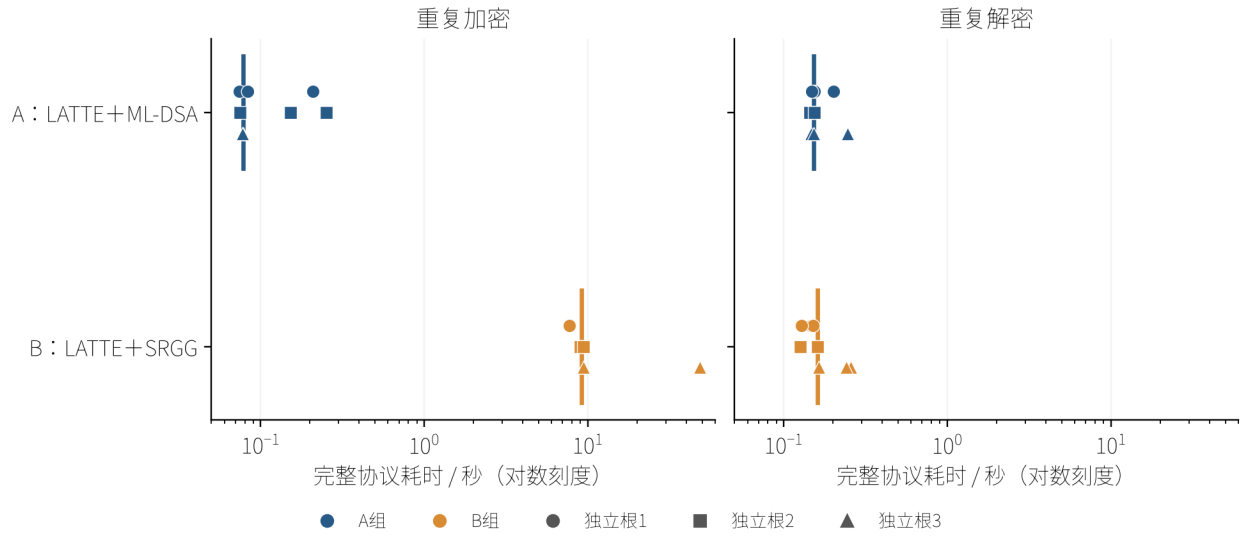


图 7.3 A/B 重复加密与解密的 9 次观测；点形区分三个独立根，短竖线标出各组中位数，横轴采用对数刻度

全部观测显示，B 有一次重复加密耗时约 48.2 秒，明显高于其中位数；A 也有较慢观测。首次解密包含叶钥生成，因此通常高于重复解密；恢复对象后，解密回到与重复操作相近的量级。

7.3.2 A/B 多根实验的对象尺寸与内存

表 7.6汇总公共对象、主状态和双方密钥的编码长度。A、B 完整密文约 139 KiB 和 146 KiB；本章后面的密文组成分析进一步解释这些字节来自哪里。

表 7.6 A/B 对象编码长度约值与运行内存

组	公钥	主状态	发送钥	接收状态	完整密文	内存/MiB
A	14	173	7.86	125	139	460
B	46.2	262	82.4	125	146	485

前五项单位为 KiB。对象长度按记录范围取近似值；内存为每 10 毫秒采样的进程树 RSS 最高值，包含运行库和子进程，短时峰值可能未被采样捕获。

B 的公钥、主状态与发送钥大于 A，接收状态接近，符合两组共用 HIBE 而采用不同 IBS 的结构。编码文件大小和运行内存分别统计：紧凑密钥文件加载后仍会恢复为数学对象，运行时还需要数值库、采样工作区和子进程。

7.3.3 C 组合的委派开销

十种组合的主实验中，C 的首次加密和解密分别为 96.4 秒和 412 秒。另一次保存了内部阶段的 32 字节消息测试记录：加密 99.0 秒，匹配解密 425 秒，其中 Lai 委派为 393 秒，占

约 92.5%。两次运行的条件和用途分别为完整运行和内部阶段分析，两份记录都表明接收端的主要瓶颈是为认可发送身份生成叶钥。该阶段分解记录的解密进程存在并发，因此只用于说明成本组成。

C 的传输密文约 4.34 MiB，大部分来自 Sageloli 签名和 Lai 胶囊。它展示了另一类 HIBE 与环 IBS 在论文配置下的完整组合，以及会话摊销的使用方式。下面分别分析通信量与准备成本。

7.3.4 Wang–PW 目标档的分阶段运行

Wang–PW 目标档使用 $n = 512$、$q = 4294967291$ 和 $s_0 = 4$。其密钥和采样工作区远大于常规组合，因此在 128 GiB 级服务器上完成目标规模运行。表 7.7汇总已经保存的建立、发钥、独立签名和 LATTE 组合结果。

表 7.7 Wang–PW 目标档与 LATTE 组合的实测结果

项目	结果	说明
Setup	536.63 s	生成目标档公开矩阵和根陷门
公开参数	67,109,371 B	规范二进制对象
主状态	603,980,295 B	保存紧凑根陷门与参数
KeyExtract	5,431.95 s	为一个 64 字节身份生成用户陷门
用户签名密钥	2,214,593,131 B	规范二进制对象
独立 Sign	175.79 s、175.15 s	两次恢复后的新消息签名均通过公开验签
签名	393,320 B	包含身份盐、消息盐和短原像
签名峰值 RSS	约 41.3 GiB	进程树采样峰值
LATTE + Wang 加密	192.54 s	78 字节应用消息，包含一次新 Wang 签名
匹配解密	2.85 s	从恢复的接收状态完成诚实解密
完整密文	529,207 B	Hybrid 与完整字节认证后的 wire 长度
组合驱动总时长	254.86 s	含准备、一次加密、诚实解密和错误输入检查

目标档实验分阶段保存并复用已经生成的根和用户密钥。Setup、KeyExtract 来自较早的目标生命周期，最终两次签名与 LATTE 组合使用修订后的签名实现；表中按阶段独立呈现各项结果。

从使用过程看，Setup 和 KeyExtract 适合放在发钥机构的离线阶段。发送端一次签名约 176 秒，并需要约 41.3 GiB 峰值内存，是组合加密的主要成本。接收端只进行公开验签与 LATTE 解封装，诚实解密约 2.85 秒。393,320 字节 IBS 签名进入对称载荷后，完整密文约 517 KiB，明显小于用户签名密钥，但仍高于 A、B。

这一结果表明通用框架能够容纳大对象矩阵 SIS 后端，同时也说明它更适合低频签名、离线发钥和资源充足的发送端。三组代表保持 A、B、C 不变，Wang–PW 作为目标规模补充

方案单独展示。

7.4 完整密文的通信开销

7.4.1 完整密文的组成

混合加密使应用消息不再受 HIBE 短明文容量直接限制，但种子胶囊、IBS 签名和一次性认证仍需要传输。依据现有安装运行记录与实际编码格式，可将 A 的一条 32 字节消息和 C 的一条空消息分解为表 7.8。

表 7.8 两条已有密文的字节组成核算

组成部分	A：消息 32 B	C：空消息
一次性公钥	65,536	65,536
一次性签名	32,768	32,768
IBS 签名	6,237	3,227,824
HIBE 胶囊	37,088	1,228,844
应用消息	32	0
上下文、格式、nonce 与标签	432	432
合计	142,093	4,555,404

整包长度来自已有运行；固定字段按序列化模块核算，HIBE 胶囊长度由整包扣除其他组成得到。该核算没有新增加密实验。

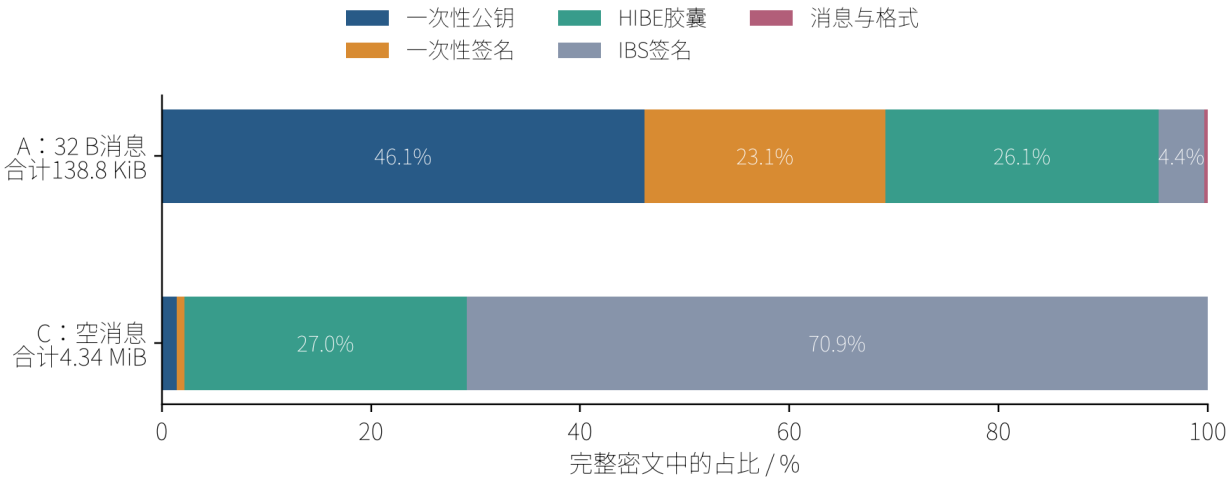


图 7.4 完整密文的主要成本来源：A 以一次性认证为主，C 以 IBS 签名和 HIBE 胶囊为主

Lamport 一次性公钥含 1,024 个 64 字节哈希像，签名含 512 个 64 字节原像，两项合计固定 96 KiB，约占这条 A 密文的 69.2%。因此，缩短几字节应用消息几乎不会改变 A 的整

包长度。C 的 IBS 签名约占 70.9%，HIBE 胶囊约占 27.0%；其后续通信优化应优先考察这两部分。

这也解释了 SRGG 签名紧凑编码与完整 B 密文的关系：签名字段变短会减少对称载荷，但整包还包含与 A 相同的一次性认证结构和 LATTE 胶囊。评价编码时应同时看单体对象与完整协议中的占比。

7.4.2 消息长度与可用容量

安装运行测试已完成 A 的 0、32、1,024 和 59,191 字节消息，B 的 0、32、1,024 和 65,364 字节消息；超过应用上限 1 字节时，系统在生成新密文前拒绝。该批最大长度密文约为 197 KiB 和 210 KiB。

C 已完成空消息和 1,024 字节消息，两条密文相差 1,024 字节，符合固定胶囊与签名加上可变应用载荷的组成。C 的 3,924 字节上限来自容量计算，尚缺该最大长度的加密解密记录。上述容量检查使用签名保守上界，具体签名和胶囊的编码长度仍可能随随机输出变化。

7.5 三项主要优化的实验结果

7.5.1 Sageloli 结构化采样结果

Sageloli 两个参数档均完成建立、身份发钥、两次签名和验证。表 7.9给出各档一套系统的运行记录。

表 7.9 Sageloli 结构化实现的实际运行，时间单位为秒

参数档	建立	发钥	首次签名	后续签名	验证	RSS/GiB
PARAM-I	4.99	85.1	9.28	2.02	0.517	1.36
PARAM-II	43.6	520	38.6	5.02	1.36	6.19

PARAM-I 发钥约 85.1 秒，记录峰值 RSS 约 1.36 GiB；PARAM-II 发钥约 520 秒，约 6.19 GiB。首次签名明显高于第二次，与同一身份陷门下数值计划可以复用的实现结构相符。两档参数都完成采样和公开验证。第五章的 1.18 TiB 是稠密矩阵表示的理论存储量估算，本表记录的是未展开该矩阵的结构化实现，因此二者分别说明原始瓶颈与实际资源占用。

7.5.2 SRGG 紧凑编码结果

图 7.5和表 7.10展示三档 SRGG 相同数学对象的两种编码。每档独立建立 3 套系统，在同一对象上交替执行两种格式的编解码。

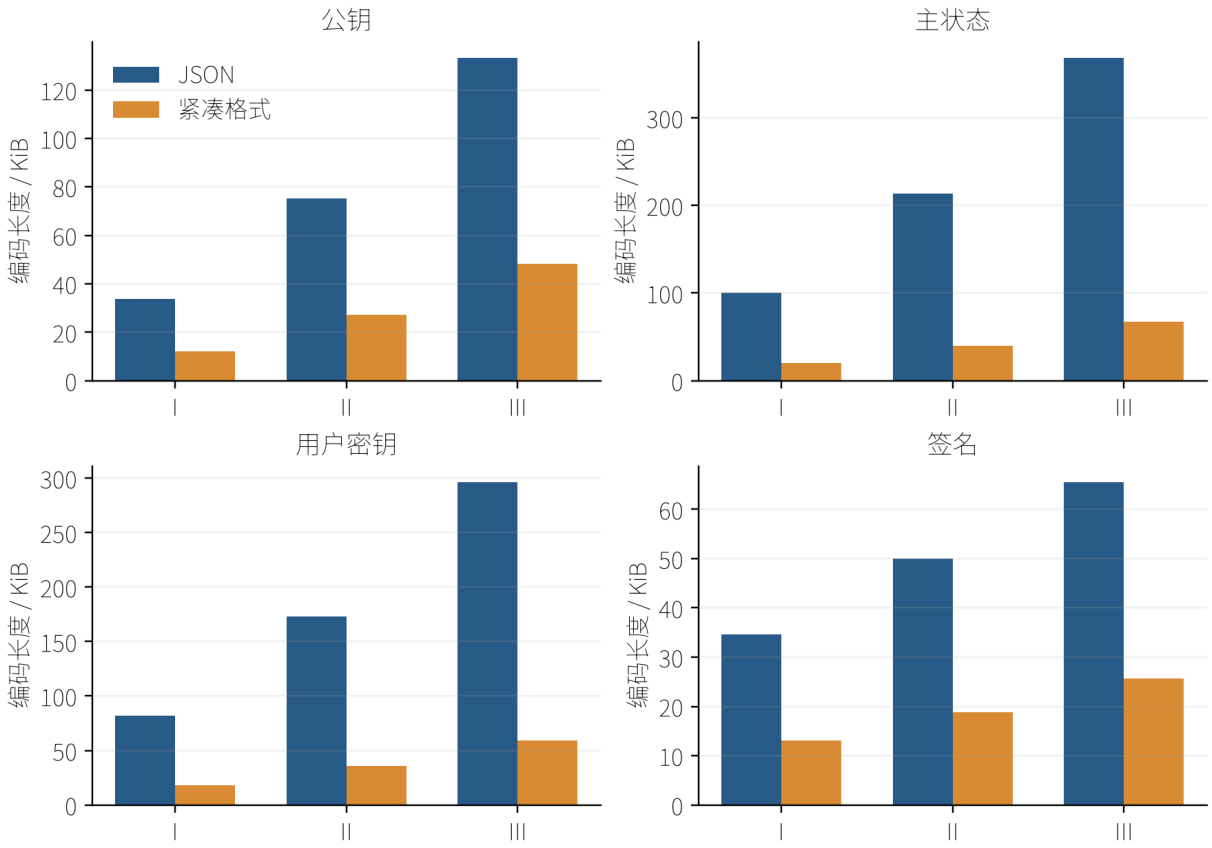


图 7.5 SRGG 三档参数的四类对象尺寸；I、II、III 表示参数档

表 7.10 SRGG 对象编码长度，单位为 KiB

参数档 / 格式	公钥	主状态	用户钥	签名
I / JSON	33.7	100	82.2	34.5
I / 紧凑	12.1	19.8	17.8	13.1
II / JSON	75.2	213	173	49.9
II / 紧凑	27.1	40.1	35.6	18.8
III / JSON	133	368	296	65.5
III / 紧凑	48.1	67.5	59.5	25.6

紧凑格式下，公钥占相应 JSON 长度的 35.9% 至 36.2%，主状态占 18.3% 至 19.8%，用户密钥占 20.1% 至 21.7%，签名占 37.7% 至 39.1%。主状态中小范围陷门系数所占比例较高，按实际取值范围位打包后，长度降幅最明显。

空间收益伴随位打包和规范检查的时间成本。以 PARAM-I 主状态为例，紧凑编码、解码中位时间约为 23.1 毫秒和 61.3 毫秒，JSON 编码、解码约为 0.974 毫秒和 37.3 毫秒。对于主状态保存、角色密钥传递和签名传输，长度下降带来的收益更直接；高频内存内转换则需要同时考虑额外处理时间。

7.5.3 Lai 接收会话结果

Lai 会话实验使用一套主密钥，原路径解密 2 次，建立会话 1 次，会话内解密 3 次。原解密中位数为 526 秒，会话准备约 510 秒，后续解密中位数为 28.6 秒；叶钥编码长度为 34.7 MiB。

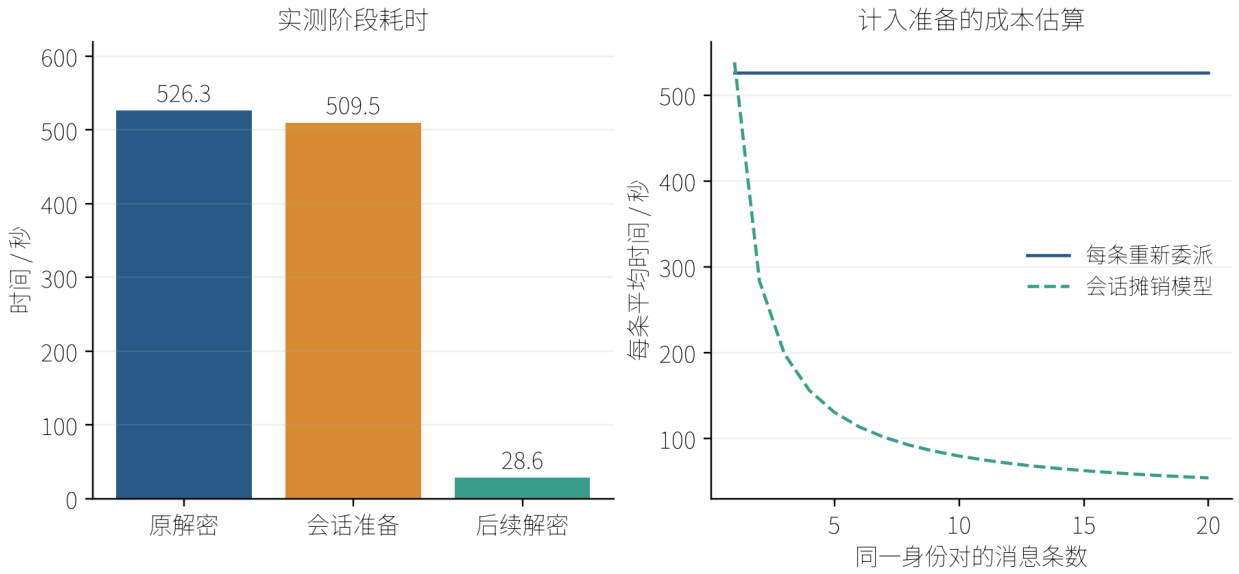


图 7.6 Lai 接收会话：左图为测得阶段耗时，右图由阶段中位数代入成本公式估算

准备完成后的单条时间约为原路径的 1/18.4。把准备计入后，首条平均成本约 538 秒；按 $\bar{T}(N)$ 模型，处理 10 条和 100 条消息时约为 79.5 秒和 33.7 秒。累计成本临界式给出 $N > 1.024$，因此从第 2 条消息起，会话模型的累计时间低于逐条委派。右图是将已有阶段中位时间代入成本公式得到的模型曲线。

会话方法把主要委派开销移到首次使用，适合固定身份对的连续消息。短暂的一次通信仍要承担完整准备；长期会话则以额外秘密状态换取较低的后续延迟。该组样本来自一套系统，后续扩大测量可进一步观察不同主密钥下的波动。

7.6 缓存与部署位置的完整流程对照

7.6.1 局部缓存对总耗时的影响

在 A、B 各 3 套系统中，分别开启和关闭优化，以相同输入执行完整流程。A 缓存机构证书的成功验证结果，原生验签调用由每条 2 次减少为 1 次，解密时间比中位数为 1.086，逐套范围 0.992 至 1.339。B 复用签名数值准备，加密时间比中位数 1.030，范围 1.009 至 1.921。

这些结果说明，减少一次局部调用后，总时间仍受其余密码运算与运行波动影响。A 有一套测量略微回退，B 的中位收益较小。因此，系统保留缓存容量与关闭选项，主要优化结

论仍以结构表示、编码和 Lai 会话为重点。

7.6.2 文件部署位置对运行时间的影响

三路部署对照固定参数、状态和原生引擎，均由同一 WSL Python 运行，分别使用 Windows 盘源码、Windows 盘安装包及 Linux 盘同一安装包。每路完成 30 次完整加密和 30 次解密，图 7.7给出中位数。

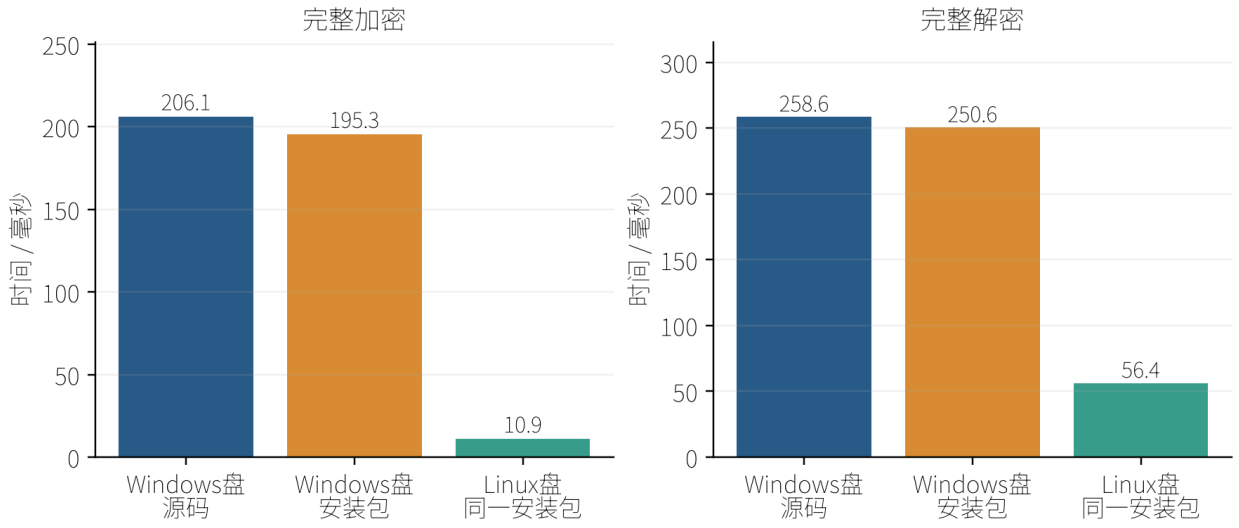


图 7.7 文件放置位置对完整 A 流程的影响；三路均使用同一 WSL 运行时

Linux 本地安装包的加密、解密中位时间分别为 Windows 盘源码运行的 5.30% 和 21.8%。排除源码与安装包差异，只比较同一安装包的两种位置时，相应比例为 5.59% 和 22.5%。剖析记录中，5 次加密产生的 515 次文件读取在挂载目录累计约 1.073 秒，在 Linux 本地约 0.011 秒，说明文件访问是该批运行的重要成本。因此，WSL 部署优先把安装包和运行状态置于 Linux 本地文件系统。

7.7 功能测试结果

十种组合的同机实验中，每组均通过 2 次诚实加密、3 次诚实解密、1 次新进程恢复，以及截断和单比特翻转两项外层拒绝，合计 20 次加密、30 次解密、10 次恢复和 20 次拒绝。更完整的异常语义由三组代表实验覆盖：三组均完成双方匹配恢复、三类身份错配和多种字节异常拒绝。表 7.11汇总主要检查；另有 A 的随机身份与消息批次，独立建立 3 套系统、测试 12 组输入，60/60 项判断符合预期。

表 7.11 三组完整协议的主要功能结果

检查情况	A	B	C
双方身份匹配	恢复原文	恢复原文	恢复原文
仅接收身份不匹配	拒绝	拒绝	拒绝
仅期望发送身份不匹配	拒绝	拒绝	拒绝
双方身份都不匹配	拒绝	拒绝	拒绝
密文字节修改、截断、追加或重复字段	拒绝	拒绝	拒绝
外层认证有效、内部签名无效	拒绝	拒绝	拒绝
新进程加载对象后解密	成功	成功	成功

身份错配测试覆盖接收者、发送者和双方同时错误，分别触发不同层次的拒绝。密文字节修改、截断、追加和重复字段覆盖外层认证与规范解析；“外层认证有效、内部签名无效”则单独检查 IBS 身份认证。新进程恢复测试重新加载公开对象和角色密钥，再对已有密文解密，覆盖持久对象的完整使用路径。

7.8 本章小结

十种常规组合在统一机器、输入和运行配置下全部完成。横向结果表明，HIBE 主要决定接收状态、胶囊长度和解密委派，IBS 主要决定发送者发钥、签名生成和发送钥尺寸。A 的在线收发处于毫秒级；B 的主要时间集中在发送端 SRGG 签名，接收端仍为亚秒级；C 的主要时间集中在 Lai 叶钥委派。Wang–PW 的 Setup、发钥和签名需要服务器级资源，但接收端解密保持在数秒量级。

通信量分析表明，A 的密文主要由一次性认证构成，C 主要由 IBS 签名和 HIBE 胶囊构成。对象级优化能否明显缩短完整密文，取决于目标字段在整包中的占比。功能测试与恢复测试进一步确认，同一套协议在 A、B、C 以及十种常规组合中保持一致的身份匹配和字节处理流程。三项主要优化分别降低了 Sageloli 采样内存、SRGG 对象尺寸和 Lai 连续解密延迟。

第八章　创新性与应用前景

本作品以“多类格原语能否组成同一套可运行 IB-ME 系统”为主线，在统一协议中实现多种 HIBE 与 IBS，并针对不同后端的主要瓶颈完成结构化计算、紧凑编码和会话复用。本章先归纳作品在通用设计、算法实现和工程优化方面的创新，再结合三组代表的性能特点分析适用场景与部署方式。

8.1　作品创新性概述

作品从协议组合、软件实现和运行优化三个层次建立完整联系。表 8.1按照“问题、方法、结果、适用条件”归纳主要工作。

表 8.1　作品主要创新及其实际效果

面向的问题	采用的方法	已实现结果	适用条件
格 HIBE 与格 IBS 的对象、参数和容量差异较大	稳定原语接口、能力声明、规范编码与组合预检	两种 HIBE、五种常规 IBS 形成十种组合，完成同机比较并接入 Wang–PW 目标档	新后端需实现接口、对象编解码、能力声明与原语自检
Sageloli 目标参数不能展开完整相关扰动矩阵	保留环多项式结构，逐频率分解低秩 Gram 矩阵	PARAM-I、II 均完成发钥、签名和公开验证，峰值 RSS 分别约 1.36、6.19 GiB	数值计划与最终精确模方程分别检查
SRGG 参考 JSON 对象含有大量文本和结构开销	按字段取值范围定长编码，并检查唯一表示	三档主状态为 JSON 长度的 18.3% 至 19.8%，签名为 37.7% 至 39.1%	以额外编解码时间换取存储和传输长度下降
Lai 固定身份对通信会重复执行二级委派	建立绑定公钥及双方身份的接收会话，复用叶密钥	准备约 510 秒后，后续解密中位数为 28.6 秒	适合固定身份对的连续消息；叶钥编码长度为 34.7 MiB
大对象矩阵 SIS IBS 难以进入统一应用流程	流式保存用户陷门，将目标档后端接入现有 LATTE 组合	完成目标规模建立、发钥、独立签名、公开验签和完整组合收发	发钥与发送端需要服务器级内存，适合低频或离线任务

第一项工作体现“通用设计”：公共五算法、传输格式、状态管理和应用入口不依赖某一种矩阵或环表示。后三项工作体现“优化实现”：先从完整流程中识别主要成本，再依据相应算法的结构改变计算或数据组织。Wang–PW 接入则进一步检验接口能否容纳远大于常规方案的密钥和签名对象。

8.2 应用需求与方案选择

IB-ME 适合“发送给某个身份，并要求只有认可特定发送身份的接收者才能恢复”的通信。与只检查接收地址的加密相比，它把发送身份条件纳入密钥恢复；与先解密再验签的简单串联相比，接收者在恢复过程中同时使用接收身份和期望发送身份。实际部署时，还要根据交互延迟、发送端资源、通信频率和状态保存条件选择具体组合。

表 8.2 代表方案与应用需求的对应关系

主要需求	建议组合	选择依据	使用条件
交互式定向消息、现场演示	A：LATTE + BNN / ML-DSA	同机重复加密、解密为 13.5、59.4 毫秒	可接受约 139 KiB 固定密文开销
原生模格身份签名与紧凑状态	B：LATTE + SRGG	同机重复加密 11.3 秒，重复解密 68.7 毫秒	发送者发钥和签名适合预先或异步执行
固定身份对的连续通信	C：Lai + Sageloli	完整重复解密 409 秒；专项会话实验降至 28.6 秒	通信双方稳定，能够保留会话叶密钥
低频、目标规模矩阵 SIS 任务	LATTE + Wang–PW	完成大对象后端的完整协议接入	发送端具备服务器级内存，密钥预先生成

该对应关系说明统一框架能够按照部署条件选择后端。应用接口、身份语义和密文字节格式保持一致，变化集中在参数、对象尺寸和运行资源。

8.3 机构定向消息与跨部门协作

在单位内部，通知、审批结果、值班指令和受控资料通常具有明确的发送部门和接收岗位。以“安全管理部门向区域值班负责人发送临时处置指令”为例，发送端以部门身份取得 IBS 签名密钥，接收端以岗位身份取得 HIBE 一级密钥。加密时指定岗位身份，接收时再指定认可的部门身份；只有两项均与密文一致时才能恢复正文。

这一模式可以减少传统证书目录查询和逐用户公钥分发的工作。人员变动时，系统可以把部门、岗位、有效期或组织层级编码进身份字符串，由发钥机构按现有组织目录签发角色对象。跨部门协作时，双方仍需共享同一套公开参数和身份编码规则，但不必在每次通信前交换临时公钥。

对于需要即时反馈的消息，A 组的在线时间更适合交互式客户端。若发送任务由后台队列执行，且希望使用原生模格 IBS 及更紧凑的状态文件，可选 B 组。系统已有命令行和浏览器入口，实际集成时还可以把规范密文字节放入现有消息队列、文件交换或数据库字段中。

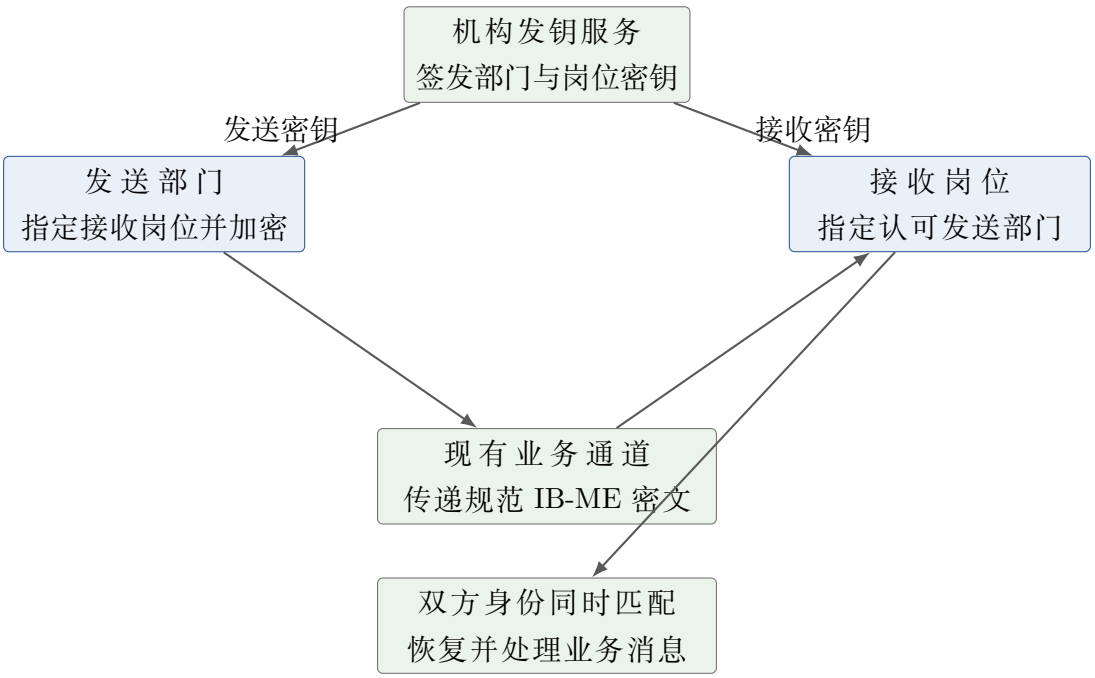


图 8.1 机构定向消息中的身份签发与密文流转

8.4 受控终端与设备指令分发

管理平台向终端设备发送配置、更新通知或控制指令时，接收方不仅要确认消息发给本设备或本设备组，还要限制可以被接受的管理身份。可以把设备编号、区域和用途组织为接收身份，把管理域或控制服务名称组织为发送身份。终端只为预先认可的管理身份派生叶密钥，从而在协议层表达“发给哪台设备、由哪个管理域发出”两个条件。

设备场景的选择取决于运行方式。偶发指令强调单次响应，可优先考虑 A 组；同一管理端持续向固定设备发送多条数据时，可以在资源允许的网关或边缘节点使用 C 组会话，将二级委派安排在连接建立阶段。对于存储受限终端，当前 Lamport 一次性认证带来的固定 96 KiB 开销仍需进一步优化，因此更适合由网关承担密码处理，而不是直接部署到极小型传感器。

8.5 固定身份对的连续通信

远程运维、跨域数据同步和周期性报送通常在较长时间内保持双方身份不变。C 组的特点正是把昂贵的 Lai 委派从每条消息移到会话准备阶段。接收端在连接建立时生成 (r, s) 叶密钥并完成绑定检查，后续每条消息继续执行字节认证、HIBE 解封装、对称认证与 IBS 验签。

这类应用应把会话生命周期与业务连接对齐：更换发送身份、接收身份、公开参数或组合配置时重新准备；连接结束后释放叶密钥；长期身份密钥仍由固定发钥服务管理。第七章的摊销模型表明，消息数量越多，单条平均成本越接近会话内解密时间。若只传送一条消息，会话并不能减少首次总成本，此时应根据延迟要求选择 A 或 B。

8.6 密码迁移与算法扩展

后量子迁移不仅是把一种算法替换成另一种算法，还要处理参数、对象格式、密文容量和运行依赖的变化。作品把这些差异放入后端能力声明和组合配置，使应用层能够在建立系统前得到身份长度、载荷容量、签名上界和依赖检查结果。新增 HIBE 可以与现有 IBS 逐一配对，新增 IBS 也可以复用现有 HIBE、五算法、状态文件、命令行和网页流程。

这种结构适合作为算法评测与迁移原型。一方面，同一完整协议可以比较不同困难问题、参数档和编码方法的实际代价；另一方面，应用可以先以低延迟组合完成业务接入，再根据部署资源切换到其他后端，而不改变双方身份含义和操作顺序。第四章的组合矩阵和 Wang–PW 接入已经展示了两种扩展方向。

8.7 应用部署的实施条件

从原型进入实际业务还需要完成身份治理、密钥保护、运行审计和统一环境测量。身份编码应与组织目录、设备台账或业务角色保持一致，并明确有效期与撤销策略；发钥机构的主状态应部署在受控环境，用户密钥应按角色分发；服务端任务需要记录组合版本、参数摘要和失败原因，但不记录主秘密或消息明文。

十种常规组合已经在 Ultra 9 275HX 上以相同源码、运行时、线程配置、身份长度和消息长度完成同机基本比较，部署选型可以先依据第七章的完整时间、对象尺寸与内存数据，再结合目标业务决定离线准备和会话复用方式。该批每种组合只有一个独立根，给出成本量级和主要瓶颈；运行波动可结合 A/B 多根实验，并在目标服务器上增加重复测量。Wang–PW 的目标参数运行使用 128 GiB 级服务器，其离线发钥和高资源发送端适合低频任务。

8.8 本章小结

作品的应用价值来自统一身份语义下的多算法选择能力。A 面向低延迟定向消息，B 面向原生模格认证和紧凑状态，C 面向固定身份对的连续通信，Wang–PW 则补充目标规模矩阵 SIS 后端。机构消息、设备指令和跨域连续通信都可以使用同一五算法与传输格式，再根据发送端资源、通信频率和状态条件选择具体组合。

第九章　总结与展望

9.1　工作总结

本作品围绕“基于多类格困难假设的抗量子身份基匹配加密通用设计和优化实现”完成了从理论构造、密码原语到应用入口的完整实现。系统以两层匿名 HIBE 承载接收身份和认可发送身份，以 IBS 认证发送者，并通过短种子混合加密承载不同长度的应用消息。完整密文进一步绑定一次性验签公钥和全部传输字节，使身份条件、载荷与外层字节认证处于同一处理流程。

在算法层，作品实现 LATTE、Lai 两种 HIBE，以及 BNN + ML-DSA、BNN + Falcon、Luo、SRGG、Sageloli 五种常规 IBS，形成十种端到端组合；在此基础上又接入 Wang–PW 目标参数矩阵 SIS 后端。三组代表分别覆盖低延迟交互、原生模格身份签名和固定身份对连续通信。所有组合复用相同的公共五算法、对象边界、状态管理、命令行和浏览器入口。

在优化层，作品针对完整运行中出现的主要瓶颈提出三项实现方法：Sageloli 结构化采样保留环结构，把大规模相关扰动转化为逐频率低秩计算；SRGG 范围感知编码按字段实际取值范围压缩四类对象；Lai 接收会话复用固定身份对的叶密钥，将重复委派移到准备阶段。三项优化都不改变相应算法的公开方程，也不要求修改通用 IB-ME 协议。

在软件层，后端通过稳定接口、能力声明和规范编解码接入。组合预检在建立系统前确认身份宽度、签名输入、载荷容量、对象格式和运行依赖；固定发钥服务与原子状态保存支持跨进程恢复；CLI 和 Web 界面完成密钥建立、角色导出、定向加密、匹配解密及错误输入演示。

9.2　实验结论

十种常规组合在 Ultra 9 275HX 统一环境下全部完成，共通过 20 次加密、30 次解密、10 次新进程恢复和 20 次外层拒绝。A 组重复加密、解密分别为 13.5 毫秒和 59.4 毫秒；B 组分别为 11.3 秒和 68.7 毫秒，额外时间集中在发送端 SRGG 签名；C 组分别为 89.4 秒和 409 秒。该批每组使用一个独立系统根，因此这些数值用于比较成本量级；多根运行波动由 A/B 专项实验另行记录。

横向比较进一步表明，HIBE 主要决定接收状态、胶囊长度和解密委派，IBS 主要决定发送者发钥、签名生成和发送钥尺寸。LATTE 五组首次解密为 95.1 至 616 毫秒，Lai 五组为 412 至 440 秒；LATTE 接收状态约 125 KiB，Lai 接收状态约 19.5 MiB。固定 IBS 后，由 LATTE 换为 Lai 使完整密文增加约 1.137 MiB；Sageloli 组合的完整密文进一步达到 3.21 和 4.34 MiB。

C 组阶段分解实验中，Lai 委派约占完整解密的 92.5%。接收会话把该阶段前移后，会话准备约 510 秒，后续解密中位数为 28.6 秒。Sageloli PARAM-I、II 均完成目标参数发钥、签名和公开验证，结构化实现的峰值 RSS 分别约 1.36 GiB 和 6.19 GiB。三档 SRGG 主状态的

紧凑长度为 JSON 的 18.3% 至 19.8%。Wang–PW 完成目标规模建立、身份发钥、独立签名、公开验签及 LATTE 组合，其大对象和服务器级资源需求也说明通用接口需要同时容纳轻量与重型后端。

密文组成分析表明，A 组主要开销来自 Lamport 一次性认证，C 组主要开销来自 Sageloli 签名和 Lai 胶囊。优化单个对象能否显著改变完整传输量，取决于该对象在整包中的占比。身份错配、字节篡改、截断、追加、内部签名错误和跨进程恢复测试则覆盖了统一协议的主要功能路径。

9.3 后续工作

第一项后续工作是扩大十种常规组合的独立根和重复次数。现有同机基本比较已经覆盖全部组合及首次、重复和恢复路径，但每组只有一个独立根；后续可在相同发布环境中增加多根试验，报告时间分位数和置信区间，并补齐 C 组 3,924 字节最大应用消息的诚实往返记录。

第二项工作是降低协议级固定通信开销。当前 Lamport 一次性公钥与签名合计 96 KiB，在 A 组短消息中占据主要比例。后续可研究更紧凑的一次性认证或直接采用具备所需强不可伪造性质的签名构造，同时保持完整密文字节绑定和现有错误输入语义。

第三项工作是继续扩展 HIBE 与 IBS 后端。接入过程已经固定为接口实现、能力声明、规范编解码、原语自检和组合测试；后续可以增加其他环格、模格或矩阵格方案，并在同机条件下比较它们在 Setup、发钥、在线收发、持久恢复和通信量方面的差异。

第四项工作是完善面向长期部署的身份与密钥管理，包括带有效期的身份编码、密钥撤销与更新、硬件隔离的主状态、会话密钥清理和运行审计。浏览器演示已经验证角色操作流程，进一步集成时可将规范密文字节接入现有消息队列或文件交换系统，并保持密码运算与业务界面的分层。

通过这些工作，系统可以从多算法研究原型继续发展为可复现、可扩展并适应不同资源条件的后量子身份基匹配加密平台。

附录 A　实验与材料复现入口

本附录给出正文图表所对应的源码、参数和公开数据入口。报告源文件位于 docs/reports/2026-10-21-review-refinement/；同目录的 build-manifest.json 记录主文件、各章、图片和 PDF 的 SHA-256 摘要，material-sources.json 记录所有生成表图的输入摘要。十种组合的实际运行源码提交为 a202c6d3c87798911675b11a46b44903405370df，源码归档摘要为 acd3f84e655a29a7fac2b755889ff5c04ac128a995bb2664bf91d09c7ff614dc。

A.1　数据、参数与源码位置

内容	仓库入口
十种组合同机比较	docs/research/2026-10-21-ten-combination-benchmark-results/；主表为basic-comparison/basic-comparison.csv，公开逐项记录位于calibration-02/ 和basic-supplement-01/
A/B 多根及通信量	docs/deliveries/2026-10-19-report-experiment-supplement/，其中master-table.json 汇总时间和对象尺寸，evidence/benchmark-A.json 与evidence/benchmark-B.json 保存逐根观测
Sageloli 结构化采样	docs/research/team-b-delivery/2026-10-19-sageloli-ring-target/；两档运行记录位于evidence/param-I-fresh.json 与evidence/param-II-fresh.json
SRGG 紧凑编码	docs/research/team-b-delivery/2026-10-18-srgg-compact-benchmark/；对象结果为wire.csv，协议载荷配对记录位于docs/research/2026-10-19-srgg-paper-latte/
Lai 接收会话	docs/research/2026-10-20-multi-backend-optimization/verified-summary.json；C 组条目记录原路径、准备和会话内解密中位时间
Wang–PW 目标档	docs/research/2026-10-19-wang-pw-target-s0-4/；公开说明、分阶段结果和验证入口见PUBLICATION_zh-CN.md 及publication/
完整参数配置	三组代表位于docs/deliveries/2026-10-19-report-three-selection/suites/；十种组合矩阵位于docs/research/2026-10-21-ten-combination-benchmark-plan/matrix.json

A.2 报告图表与 PDF 重建

在仓库根目录使用本机配置的 Python 与 Tectonic 运行以下命令；中文字体路径只作为命令参数传入，不写入报告源文件。

```
git ibme-run python3 docs/reports/2026-10-21-review-refinement/\
prepare_materials.py --font <中文字体路径>
git ibme-run python3 docs/reports/2026-10-21-review-refinement/\
build_pdf.py
git ibme-run python3 docs/reports/2026-10-21-review-refinement/\
check_report.py --require-pdf
```

第一条命令从公开 JSON 和 CSV 重建九份数据表、八张数据图、密文字段核算与来源摘要，不执行新的密码运算。第二条命令调用 Tectonic 生成 PDF 并写入输入摘要。第三条命令检查父稿未被修改、章节与引文结构、数据绑定、A4 页面和编译日志；逐页视觉复核另记录在 `visual-review.json`。

十种组合的公开结果可由比较工具重新汇总。下列命令只读取已归档记录，并把结果写入新的输出目录：

```
git ibme-run python3 -m experiments.research.\
ten_combination_basic_comparison \
  --design docs/research/2026-10-21-ten-combination-benchmark-results/\
basic-design.json \
  --reused-public docs/research/2026-10-21-ten-combination-benchmark-results/\
calibration-02 \
  --supplemental-public docs/research/2026-10-21-ten-combination-benchmark-results/\
basic-supplement-01 --output <新输出目录> --plots
```

真实重跑入口为`experiments/research/ten_combination_benchmark.py` 及其工作进程；运行时应重新记录源码提交、原生组件、依赖、线程和硬件信息，并使用新的私有状态目录。正文中的时间数据绑定上述已归档批次，重建表图与重新执行密码实验是两个独立步骤。

参考文献

[1] Peter W. Shor. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.

[2] Adi Shamir. Identity-Based Cryptosystems and Signature Schemes. In *Advances in Cryptology—CRYPTO 1984*, LNCS 196, pp. 47–53, 1985.

[3] Dan Boneh and Matthew Franklin. Identity-Based Encryption from the Weil Pairing. *SIAM Journal on Computing*, 32(3), pp. 586–615, 2003.

[4] Craig Gentry and Alice Silverberg. Hierarchical ID-Based Cryptography. In *ASIACRYPT 2002*, LNCS 2501, pp. 548–566, 2002.

[5] Florian Hess. Efficient Identity Based Signature Schemes Based on Pairings. In *Selected Areas in Cryptography 2002*, LNCS 2595, pp. 310–324, 2003.

[6] Mihir Bellare, Chanathip Namprempre, and Gregory Neven. Security Proofs for Identity-Based Identification and Signature Schemes. In *EUROCRYPT 2004*, LNCS 3027, pp. 268–286, 2004.

[7] Miklós Ajtai. Generating Hard Instances of Lattice Problems. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pp. 99–108, 1996.

[8] Oded Regev. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing*, pp. 84–93, 2005.

[9] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for Hard Lattices and New Cryptographic Constructions. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, pp. 197–206, 2008.

[10] David Cash, Dennis Hofheinz, Eike Kiltz, and Chris Peikert. Bonsai Trees, or How to Delegate a Lattice Basis. In *EUROCRYPT 2010*, LNCS 6110, pp. 523–552, 2010.

[11] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient Lattice (H)IBE in the Standard Model. In *EUROCRYPT 2010*, LNCS 6110, pp. 553–572, 2010.

[12] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Lattice Basis Delegation in Fixed Dimension and Shorter-Ciphertext Hierarchical IBE. In *CRYPTO 2010*, LNCS 6223, pp. 98–115, 2010.

[13] Markus Rückert. Strongly Unforgeable Signatures and Hierarchical Identity-Based Signatures from Lattices without Random Oracles. In *PQCrypto 2010*, LNCS 6061, pp. 182–200, 2010.

[14] Giuseppe Ateniese, Danilo Francati, David Nuñez, and Daniele Venturi. Match Me If You Can: Matchmaking Encryption and Its Applications. In *CRYPTO 2019*, LNCS 11693, pp. 701–731, 2019.

[15] Danilo Francati, Alessio Guidi, Luigi Russo, and Daniele Venturi. Identity-Based Matchmaking Encryption without Random Oracles. In *INDOCRYPT 2021*, LNCS 13143, pp. 415–435, 2021.

[16] Jie Chen, Yu Li, Jinming Wen, and Jian Weng. Identity-Based Matchmaking Encryption from Standard Assumptions. In *ASIACRYPT 2022*, LNCS 13793, pp. 394–422, 2022.

[17] Yuejun Wang, Baocang Wang, Qiqi Lai, and Yu Zhan. Identity-based Matchmaking Encryption with Stronger Security and Instantiation on Lattices. *Theoretical Computer Science*, 1029:115048, 2025. doi: 10.1016/j.tcs.2024.115048.

[18] Sohto Chiku, Keitaro Hashimoto, Keisuke Hara, and Junji Shikata. Identity-Based Matchmaking Encryption with Enhanced Privacy Against Chosen-Ciphertext Attacks. Cryptology ePrint Archive, Paper 2023/1435, 2023, revised 2026.

[19] Roberta Cimorelli Belfiore, Andrea De Cosmo, and Anna Lisa Ferrara. Identity-Based Matchmaking Encryption from Standard Lattice Assumptions. In *ACNS 2024*, LNCS 14584, pp. 163–188, 2024.

[20] Rishab Goyal, Venkata Koppula, and Mahesh Sreekumar Rajasree. A Note on Adaptive Security in Hierarchical Identity-Based Encryption. In *CRYPTO 2025*, LNCS 16002, pp. 101–136, 2025.

[21] Qing Yu, Shengli Liu, and Yiming Li. Identity-Based Signature and Identity-Based Matchmaking Encryption from Lattices. *Theoretical Computer Science*, 1082:116152, 2026. doi: 10.1016/j.tcs.2026.116152.

[22] Raymond K. Zhao, Sarah McCarthy, Ron Steinfeld, Amin Sakzad, and Máire O'Neill. Quantum-safe HIBE: Does It Cost a LATTE? *IEEE Transactions on Information Forensics and Security*, 2023.

[23] Russell W. F. Lai, Henry K. F. Cheung, and Sherman S. M. Chow. Trapdoors for Ideal Lattices with Applications. In *Inscrypt 2014*, LNCS 8957, pp. 239–256, 2015. doi: 10.1007/978-3-319-16745-9_14.

[24] Huiwen Jia, Ying Liu, Chunming Tang, and Lin Wang. Constructing Efficient Identity-Based Signatures on Lattices. *IET Information Security*, 2025. doi: 10.1049/ise2/6684889.

[25] Éric Sageloli, Pierre Pébereau, Pierrick Méaux, and Céline Chevalier. Shorter and Faster Identity-Based Signatures with Tight Security in the (Q)ROM from Lattices. In *ACNS 2023*, 2023. doi: 10.1007/978-3-031-33488-7_24.

[26] Jiaxin Pan and Benedikt Wagner. Short Identity-Based Signatures with Tight Security from Lattices. In *PQCrypto 2021*, 2021. ePrint 2021/970.

[27] David Galindo, Javier Herranz, and Eike Kiltz. On the Generic Construction of Identity-Based Signatures with Additional Properties. In *ASIACRYPT 2006*, 2006.

[28] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, 2024.

[29] Daniele Micciancio and Chris Peikert. Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller. In *EUROCRYPT 2012*, 2012.

[30] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. *Journal of the ACM*, 60(6), 2013.

[31] Adeline Langlois and Damien Stehlé. Worst-Case to Average-Case Reductions for Module Lattices. *Designs, Codes and Cryptography*, 75, pp. 565–599, 2015.

[32] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.

[33] National Institute of Standards and Technology. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. Special Publication 800-38D, 2007.

[34] Leslie Lamport. Constructing Digital Signatures from a One Way Function. SRI International, CSL-98, 1979.

[35] Pingbin Luo, Xinjian Chen, Willy Susilo, and Qiong Huang. Identity-Based Signature from Lattices without Trapdoors. In *ICICS 2024*, 2024.

[36] Pierre-Alain Fouque et al. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. Specification v1.2, 2020.